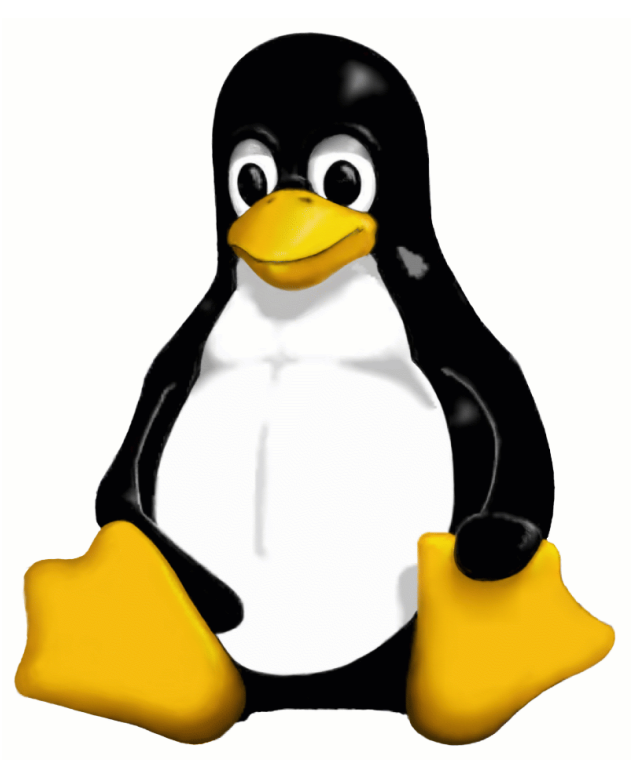
Hochschule Esslingen
University of Applied Sciences

Linux-Crashkurs

Lerntext zur Verwendung im Modul *Betriebssysteme*



Prof. Dr. Peter Väterlein
Hochschule Esslingen

Version 2.2
Stand: März 2016

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

*Dieses Skript „Linux Crashkurs“ darf in seiner Gesamtheit **nur zum privaten Studiengebrauch** benützt werden. Das Skript ist in seiner Gesamtheit urheberrechtlich geschützt. Folglich sind Vervielfältigungen, Übersetzungen, Mikroverfilmungen, Scan-Vervielfältigungen, Verbreitungen, sowie die Einspeicherung und Verarbeitung in elektronischen Systemen unzulässig. Ein darüber hinausgehender Gebrauch ist straf- und zivilrechtlich unzulässig.*

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Inhaltsverzeichnis

1 Vorbemerkung 7
2 Konventionen 8
2.1 Typographie 8
2.2 Systemvoraussetzungen 8
3 Aufsetzen einer Testumgebung 9
3.1 Live CD/DVD 9
3.2 Virtualisierung – Rechner im Rechner 10
3.3 The Real Thing – Linux installieren 10
4 First Contact – Der Login 12
5 Die Bourne Again Shell (bash) 14
5.1 Kommandos ausführen 16
6 Wichtige UNIX Kommandos 17
6.1 Hiiiilfeeee! 17
6.2 Verzeichnisse 21
6.3 Dateien 29
6.4 Umlenkung von Ein- und Ausgabe 40
6.5 Dateisysteme 46
6.6 Änderungen am eigenen Benutzerkonto 49
7 Reguläre Ausdrücke 52
8 Textfilter unter UNIX 55
8.1 Textdateien sortieren 55
8.2 Suchen nach Strings 57
8.3 Texte filtern 58
9 Bash für Profis 72
9.1 Spielarten von Shells 72
9.2 Startup-Dateien 74
9.3 Jobkontrolle 77
9.4 Shell History 80
10 Shellskripte 84
10.1 `Mein erstes Shellskript` 84
10.2 Variable 86
10.3 Arithmetische Ausdrücke 88
10.4 Verkettung von Anweisungen 89
10.5 Logische Ausdrücke 89
10.6 Bedingte Verzweigungen 91
10.7 Schleifen 93
10.8 Shell-Funktionen 95
11 Antworten auf die Testfragen 96
12 Index 100

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

1 Vorbemerkung

Im September 1991 veröffentlichte ein finnischer Informatikstudent namens Linus Torvalds die Version 0.01 eines Betriebssystem-Kernes in einem Diskussionsforum des Internets[1]. Er nannte das neue Betriebssystem *Linux*, was gelegentlich als Abkürzung von *Linux is not UNIX* interpretiert wird. Denn obwohl Linux große Ähnlichkeit mit UNIX hat, ist es doch eine völlig neue Implementierung, die in keinem Teil auf den Sourcecode des originalen UNIX zurückgreift[2]. Der Quellcode von Linux unterlag von Anfang an der „GNU General Public License“[3] und stand damit jedermann zur freien Verfügung.

In der Zwischenzeit hat Linux einen weltweiten Siegeszug angetreten und läuft heute in Mikrocontrollern und Smartphones ebenso wie in großen Webservern und in Personalcomputern ebenso wie in den schnellsten Rechnern der Welt mit Zigtausenden von Prozessoren[4]. In der akademischen Ausbildung wird Linux gerne als Prototyp für ein UNIX-artiges Betriebssystem herangezogen. Da der vollständige Quelltext verfügbar ist, kann die Funktion bis ins Detail untersucht und demonstriert werden.

Auch wenn Microsoft weiterhin den Markt für Desktop-Betriebssysteme dominiert, sind Grundkenntnisse in der Bedienung von Linux bzw. UNIX wegen seiner vielfältigen Anwendungsbereiche in der Informationstechnik unabdingbar für eine Ingenieurin oder einen Ingenieur der Informationstechnik. Deshalb sind in diesem Lerntext die wichtigsten Grundlagen der Linux-Bedienung zusammengefasst.

Das Skript ist ein Anfang und wird im Laufe des Semesters durch weiteren Stoff im Rahmen der Vorlesung ergänzt. Es ist dazu gedacht, dass die Studierenden es aktiv durcharbeiten. Das heißt, die Beispiele sollten gleich am Rechner nachvollzogen werden. Denn:

---

[1] Wie sich das damals zugetragen hat, kann man in einem sehr unterhaltsamen Buch nachlesen: Linus Torvalds und David Diamond, *„Just for Fun“*, dtv Verlag (2002).

[2] Die Frage, ob und in wieweit Linux originalen UNIX Source-Code enthält, beschäftigt seit Jahren die US-Gerichte. Bisher konnten die Linux-Vertreter aber immer die Originalität des Linux-Sourcecodes nachweisen.

[3] http://www.gnu.org/licenses/gpl.html

[4] Diese Rechner werden zweimal pro Jahr ermittelt und in der „TOP500-Liste“präsentiert (http://www.top500.org)

Der Mensch behält ca. 30 % von dem, was er sieht, aber über 90 % von dem, was er selber tut!

2 Konventionen

2.1 Typographie

Zur Kennzeichnung bestimmter Textteile werden folgende typographische Konventionen verwendet

- Die Namen von Kommandos und Dateien, sowie Pfadangaben werden `in Schreibmaschinenschrift` gedruckt
- Ausgaben von Computern in den Übungsbeispielen werden `in Schreibmaschinenschrift mit blauem Hintergrund` gedruckt
- Benutzereingaben in den Übungsbesipielen werden in **`halbfetter Schreibmaschinenschrift mit blauem Hintergrund`** gedruckt

2.2 Systemvoraussetzungen

Um die Übungen in diesem Skript machen zu können, benötigt man einen laufenden Linux-Rechner. Prinzipiell ist es egal, was das für ein Rechner ist und auch die Wahl der Linux-Distribution ist frei. Zum Testen der Anleitungen und Aufgaben wurde *Fedora Core 23* verwendet. Wenn kein Rechner zur Verfügung steht, auf dem Linux installiert werden kann, wäre die Verwendung eines „LiveCD"-Systems eine mögliche Alternative. Solche CD- oder DVD-Images bieten inzwischen fast alle gängigen Distributionen an. Aber natürlich kann zum Üben auch jeder Rechner im Betriebssysteme-Labor oder in den Linux-Pools des Rechenzentrums verwendet werden.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

3 Aufsetzen einer Testumgebung

Es gibt verschiedene Möglichkeiten, eine Umgebung zu installieren, mit dem die Übungen in diesem Lerntext durchgeführt werden können.

3.1 Live CD/DVD

Live CD Varianten gibt es von fast allen gebräuchlichen Linux-Distributionen. Auf dem FTP-Server der Hochschule Esslingen ist eine Vielzahl verschiedener Varianten verfügbar. Die Live-CD bzw. -DVD-Images für die Distribution Fedora 23 liegen unter dem Pfad

KNOPPIX: der einfachste Weg zu einem funktionierenden Linux System

```
http://ftp-stud.hs-esslingen.de/Mirrors\[5]
   /fedora.redhat.com/linux/releases/23/Live/x86_64
```

Das Image kann auf einen PC mit CD- bzw. DVD-Brenner heruntergeladen und dann auf einen Rohling gebrannt werden. Mit der so gebrannten CD/DVD kann dann ein beliebiger PC gebootet werden.

Im einfachsten Fall kann man am Bootprompt einfach die ENTER-Taste drücken. Am Ende des Startvorgangs befindet man sich auf einer grafischen Oberfläche, (meist „KDE“ oder „Gnome“), ohne dass ein Login-Vorgang notwendig gewesen wäre. Der Name des angemeldeten Benutzers ist knoppix.

Wenn sich Ihre Tastatur seltsam verhält, liegt das vielleicht daran, dass Sie eine englische Version von Knoppix erwischt haben. In diesem Fall müssen Sie am Bootprompt noch die Option lang=de eingeben (das „=“ Zeichen liegt bei der englischen Tastatur auf der Taste links neben der Backspace Taste („´“ und „`“)).

Es gibt noch eine Reihe weiterer Optionen. Wenn am Bootprompt die Tasten F2 bzw. F3 gedrückt werden, werden die verschiedenen Möglichkeiten aufgezählt.

---

[5] Der „Backslash“ oder „Rückschräger“ („\“) signalisiert bei Linux/Unix häufig, dass der folgende Zeilenumbruch ignoriert werden soll und die nächste Zeile eigentlich noch zu dieser gehört.

Der größte Nachteil der Live CD/DVD ist, dass keine benutzerspezifische Daten gespeichert werden. Mit dem OpenSource Werkzeug LinuxLive USB Creator können Sie einen bootfähigen USB Stick aus einer beliebigen Linux Live CD erzeugen.

3.2 Virtualisierung – Rechner im Rechner

Virtualisierung: Den Kuchen essen und behalten – Linux und Windows gleichzeitig auf einem Rechner

Die zweite Möglichkeit, ein funktionierendes Linux-System zu bekommen, ohne den eigenen Rechner neu zu installieren, ist die Verwendung einer Virtualisierungssoftware.

Kostenlos im Internet erhältlich ist die Software *VirtualBox* (`www.vitualbox.org`), die ursprünglich von der Firma Sun Microsystems (heute Oracle) entwickelt wurde. Diese Software läuft unter Microsoft Windows, Apple OS X, Linux und Sun Solaris. Sie stellt einen virtuellen PC zur Verfügung, auf dem man grundsätzlich alle Betriebssysteme installieren kann, die auf einem PC laufen.

Für Studierende und Mitarbeiter/-innen der Hochschule Esslingen ebenfalls kostenlos sind die Produkte der Firma VMWare. Man bekommt sie über die Intranet-Seiten des Rechenzentrums.

Wenn die Virtualisierungssoftware installiert ist, kann man entweder eine fertige virtuelle Maschine aus dem Internet herunterladen (auf der VMware Webseite nach „Virtual Appliance“ suchen). Oder man installiert eine neue virtuelle Maschine. Dazu wird dann noch ein Installationsmedium des gewünschten Betriebssystems benötigt. Für verschiedene Linux-Distributionen findet man entsprechende Image-Dateien auf dem FTP-Server der Hochschule.

Es ist nicht nötig, die Installationsimages der gewünschten Linux-Distribution auf eine CD bzw. DVD zu brennen. Sowohl *VirtualBox* als auch *VMWare* erlauben es, ISO9660 Images (`.iso` Dateien) als virtuelle CD-Laufwerke zu nutzen.

3.3 The Real Thing – Linux installieren

Linux installieren: die performanteste Lösung

Um Linux tatsächlich auf einem PC zu installieren, legt man einfach eine entsprechende Installations-CD/DVD in das Laufwerk des PC ein und startet das System von diesem Medium. Möchte man Windows und Linux parallel auf einem Rechner installieren, muss man zunächst Windows installieren, wobei man sinnvollerweise etwas Platz

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

auf der Festplatte frei lässt (minimal 10 GB). Ist Windows bereits installiert, kann man das Dateisystem verkleinern[6], um so Platz für Linux zu schaffen. Das ist allerdings nicht ohne Risiko und sollte nicht bei einem System gemacht werden, das wichtige Daten enthält.

Ein so installiertes System ist leistungsfähiger als die bisher genannten Möglichkeiten. Für die Übungen in diesem Skript ist das allerdings unerheblich.

Bei der Installation von Fedora-basierten Systemen werden Sie während der Installation gefragt, ob Sie neben dem Administratorkonto root noch ein weiteres Benutzerkonto einrichten wollen. Das sollten Sie unbedingt tun. Vor allem unerfahrene Benutzer/-innen können als „root“ sehr schnell großen Schaden an ihrem System anrichten. Als normaler Benutzer kann das nicht passieren.

Wenn Sie das Konto mit der Option „Systemadministrator“ einrichten, dann können sie volle administrative Rechte auf Ihrem System erhalten, wenn Sie das Kommando sudo verwenden.

Auf Ubuntu-basierten Systemen wird das root-Konto normalerweise gar nicht direkt benutzt, sondern immer mit dem sudo-Kommando gearbeitet, wenn administrative Rechte notwendig sind.

---

[6] Dafür kann man kommerzielle Werkzeuge wie Partition Magic einsetzen. Manche Linux-Distributionen (z.B. OpenSuSE 10.2) bieten diese Option bei der Installation von sich aus an.

4 First Contact – Der Login

Linux- bzw. UNIX-Rechner sind traditionell für den Einsatz in Computernetzen und für die Bedienung durch mehrere Benutzer konzipiert. Deshalb ist es notwendig, dass sich jede Person, die den Rechner nutzen möchte, zunächst identifiziert. Dies geschieht durch Eingabe eines Benutzernamens und eines Passwortes.

UNIX Benutzernamen und Passwörter dürfen nicht mehr als acht Zeichen habe

Bei traditionellen UNIX-Systemen ist die Länge von Benutzername und Passwort auf acht Zeichen beschränkt, wobei man sich tunlichst auf Standard-ASCII-Zeichen beschränken sollte. Bei aktuellen Linux-Systemen besteht diese Einschränkung allerdings nicht mehr.

Bei den meisten Linux-Rechnern wird heute beim Systemstart automatisch die graphische X-Windows-Oberfläche gestartet[7]. Auf dem Begrüßungs-Bildschirm findet sich ein Feld, in dem man seinen Benutzernamen angeben kann. Oder es wird eine Liste von gültigen Benutzernamen angezeigt. Das Feld für das Passwort ist unter Umständen erst sichtbar, wenn man den Benutzernamen eingegeben hat.

Neben der graphischen Oberfläche haben Linux-Rechner in der Regel noch eine oder mehrere Textkonsolen, deren Funktionalität den früher in der UNIX-Welt verbreiteten Terminals entspricht. Der Vorteil der Textkonsolen ist ihr geringer Bedarf an Rechner-Ressourcen. Viele Server-Systeme verzichten daher auf die X-Windows-Oberfläche.

Umschalten zu den Textkonsolen: [Strg][Alt][F*n*].

Die Textkonsolen erreicht man durch gleichzeitiges Drücken der [Strg]-, der [Alt]- und einer Funktionstaste. Bei älteren Linuxvarianten erreicht man mit den Tasten [F1] bis [F6] Textkonsolen, während man mit der Tastenkombination [Strg][Alt][F7] zurück zur graphischen Oberfläche gelangt. Bei neueren Linux-Distributionen liegt die gra-

---

[7] Aktuelle Linux-Distributionen bieten mehrere Varianten des X-Windows-Systems, so genannte Desktops, an. Die populärsten davon sind KDE und Gnome. Welcher Desktop standardmäßig verwendet wird, hängt von der gewählten Distribution ab. Auf dem Anmeldebildschirm wird meist eine Auswahl angeboten, um die bevorzugte Umgebung beim Login zu starten. Bei der Fedora Core Distribution heißt dieser Button „Sitzung“. Für die Beispiele in diesem Skript wurde KDE verwendet. Wer lieber Gnome, Xfce oder den guten alten mwm verwendet, kann das aber gerne tun. Unter Umständen müssen dann einige Erklärungen sinngemäß übertragen werden.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

phische Oberfläche auf [Strg][Alt][F1], während die Textkonsolen über die Funktionstasten [F2] und folgende erreicht werden.

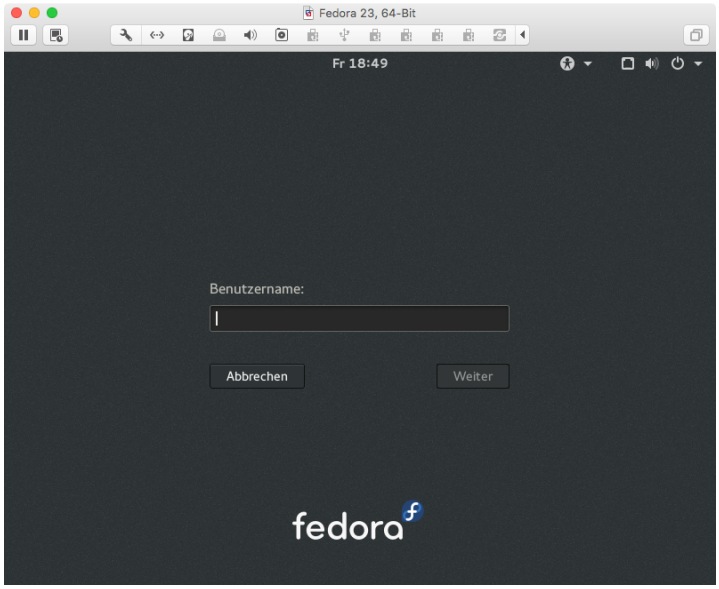


Abbildung 4.1: *Anmeldebildschirm der graphischen Oberfläche von Linux (hier: Fedora 23 als virtuelle Maschine in VMWare Fusion)*

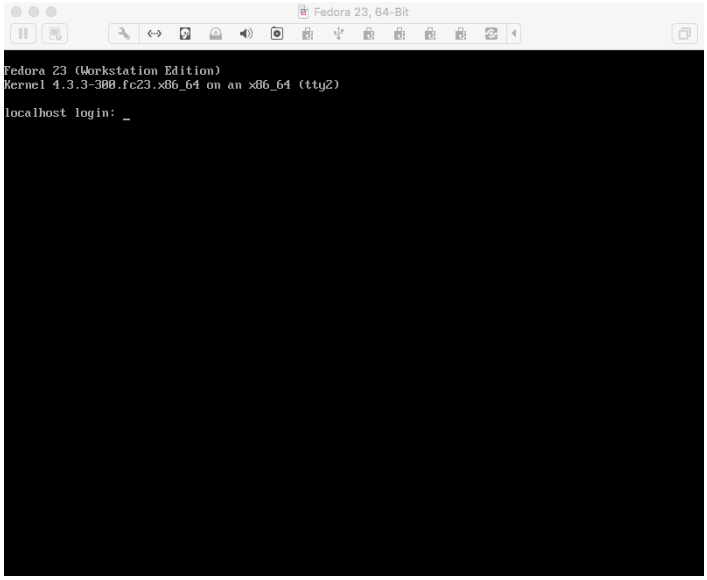


Abbildung 4.2: *Anmeldebildschirm auf der Textkonsole von Linux (hier: Fedora 23 als virtuelle Maschine in VMWare Fusion)*

Nach der Anmeldung an einer Textkonsole landet man sofort auf der Kommandozeile, die bei UNIX-Systemen als *Shell* bezeichnet wird und die für die Erforschung der UNIX Kommandos in diesem Skript benö-

tigt wird. Bei der graphischen Oberläche muss man sich zunächst eine virtuelle Konsole beschaffen, das in einem Fenster auf der graphischen Oberfläche die Funktionen eines textbasierten Terminals bietet.

Je nach Distribution und Desktop findet man die Terminal-Anwendung an verschiedenen Stellen. Manchmal ist ein entsprechendes Icon (Schwarzes Rechteck mit hellem Rand) schon in der Kopf- oder Fußzeile vorhanden. Oder man findet es im „Startmenue“. Bei der aktuellsten Gnome-Oberfläche, findet man ein solches Programm, wenn man in der linken oberen Bildschirmecke auf „Aktivitäten“ klickt und in das Suchfeld (das mit der Lupe) „Terminal“ eingibt.

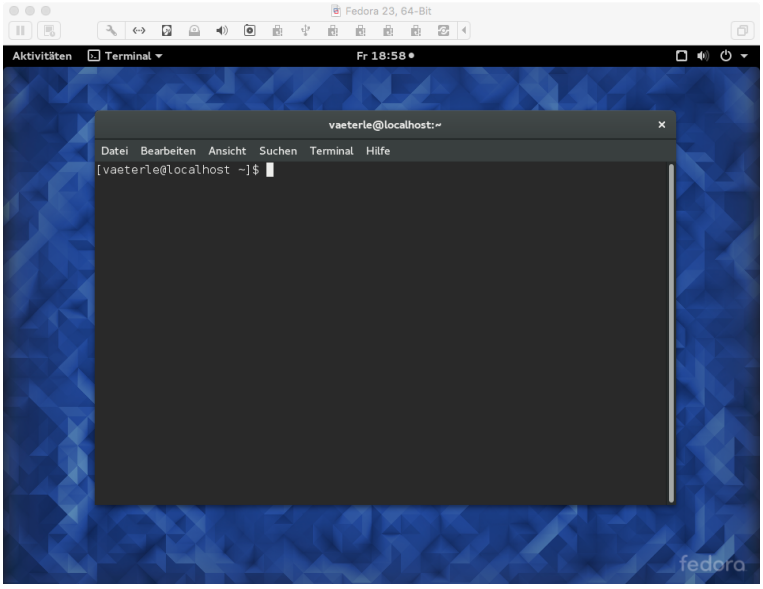


Abbildung 4.3: *Gnome-Oberfläche (CentOS 6.4) mit geöffnetem virtuellem Terminal*

5 Die Bourne Again Shell (bash)

Bei modernen Computersystemen benutzt man in der Regel graphische Benutzerschnittstellen wie MS Windows oder das X-Windows-System auf UNIX/Linux-Rechnern. Solche *Graphical User Interfaces (GUI)* sind oft intuitiv benutzbar und erlauben auch dem unerfahrenen Benutzer einen unkomplizierten Zugang zum Rechner und seinen Anwendungsprogrammen.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Aber so nützlich eine graphische Benutzerschnittstelle für den Anfänger oder den gelegentlichen Benutzer ist, so lästig ist sie oft für den Profi. Denn auch, wenn er oder sie genau weiß, was zu tun ist, um das gewünschte Ziel zu erreichen, wird das Procedere nicht einfacher. Man muss sich immer durch alle Schritte durchklicken. Deshalb werden insbesondere im Bereich der Administration von Rechnern und Rechnernetzen häufig textbasierte Werkzeuge verwendet, die von einer Kommandozeile aus gestartet werden können. Auf diese Weise kann man oft mit einem einzigen Befehl und geeigneten Parametern in Sekunden erreichen, wozu man mit graphischen Werkzeugen Minuten gebraucht hätte. Vor allem lassen sich solche Kommandos in Skript-Dateien zusammenfassen. Auf diese Weise lassen sich Standard-Aufgaben der Administration automatisieren.

Auf kleinen Linux-Systemen, zum Beispiel auf Mikrocontrollern, verbietet sich eine graphische Schnittstelle schon aus Gründen der Ressourcenknappheit. Hier ist die Kommandozeile der einzige Zugang zum Rechner.

Shells sind Kommandozeileninterpreter, das heißt Programme, die Benutzereingaben als Kommandos interpretieren und diese ausführen

Aber damit man diese Profiwerkzeuge benutzen kann, braucht man ein Programm, das Eingaben der Tastatur auswertet, die entsprechenden Kommandos startet und die Resultate auf einem Bildschirm oder in einem Fenster der graphischen Oberfläche ausgibt. Ein solches Programm heißt *Komandozeileninterpreter* oder in der UNIX-Welt *Shell* [8].

Auf Linux-Systemen hat sich eine Implementierung der originalen UNIX-Shell durchgesetzt, deren Quelltext frei verfügbar ist und die derselben GNU General Public License unterliegt, wie der Linux Kernel. Da die ursprüngliche UNIX Shell (Kommando „sh“) von Stephen Bourne geschrieben wurde, heißt diese neue Implementierung *Bourne Again Shell* oder kurz *bash*.

---

[8] vom englischen Wort für *Hülle* oder *Außenhaut*. Ursprünglich war die Shell die äußerste, dem Benutzer zugängliche, Schicht des Betriebssystems. Insofern könnte man die GUI's moderner Computersysteme auch als Shells bezeichnen.

5.1 Kommandos ausführen

Probieren geht über Studieren! Sie sollten die blau unterlegten Beispiele unbedingt ausprobieren. Denn der Mensch behält 30% von dem, was er sieht, aber 90% von dem, was er selber tut.

Ein UNIX Kommando auszuführen bedeutet, die Shell anzuweisen, eine ausführbare Datei gleichen Namens zu starten. Dabei muss der Name dieser Datei keine besonderen Endungen wie „.EXE“ oder „.COM“ aufweisen.

Um die Dateien und Unterverzeichnisse des aktuellen Verzeichnisses anzuzeigen, müsste man eigentlich eintippen

```
bash> /bin/ls
```

da die Datei `ls` sich im Verzeichnis `/bin`[9] befindet. Allerdings kann man der Shell mitteilen, welche Verzeichnisse standardmäßig nach einem eingegebenen Kommando durchsucht werden sollen. Wie bei DOS geschieht das durch die Definition einer Umgebungsvariable namens $PATH, den so genannten Suchpfad.

Welche Verzeichnisse im Suchpfad enthalten sind, erfährt man mit dem Kommando

```
bash > echo $PATH
/bin:/usr/bin:/usr/local/bin:/etc:/usr/X11R6/bin
bash >
```

Das Verzeichnis `/bin` gehört also zum Suchpfad. Um das Kommando aufzurufen, reicht es daher, `ls` einzugeben.

Das Kommando echo ist übrigens im Gegensatz zu `ls` eines der eingebauten Kommandos der bash. Um dieses Kommando auszuführen, wird keine weitere ausführbare Datei benötigt[10].

---

[9] UNIX Verzeichnisse sind wie bei MS DOS bzw. MS Windows hierarchisch angeordnet. Die verschiedenen Ebenen werden allerdings durch „Slashes“ („/“) getrennt und nicht durch „Backslashes“ („\“). Die Entwickler von DOS haben das hierarchische Dateisystem von UNIX nachempfunden, allerdings mit einem anderen Trennzeichen, um urheberrechtlichen Ärger zu vermeiden.

[10] Da nicht alle UNIX Shells ein eingebautes „echo“-Kommando haben, gibt es auch eine ausführbare Datei `/bin/echo`. In solchen Fällen bevorzugt die bash allerdings die eigene, eingebaute Variante eines Kommandos. Möchte

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

6 Wichtige UNIX Kommandos

Die allermeisten UNIX-Kommandos sind sogenannte Dienstprogramme, das heißt, ausführbare Dateien, die von einer Shell oder einem anderen Programm gestartet werden können. Daneben gibt es die bereits erwähnten eingebauten Kommandos der verschiedenen Shells.

Schon bevor Linux entwickelt wurde, war es ein Ziel der *Free Software Foundation*[11], Open-Source Pendants für die wichtigsten UNIX-Kommandos zu schreiben. Diese so genannte „*GNU-Software*“ (GNU = „GNU is not UNIX“) wurde unter der *GNU General Public License* (*GPL*) jedermann frei zugänglich gemacht und sollte schließlich zu einem Open-Source Betriebssystem nach UNIX-Art weiterentwickelt werden. Die Entwicklung von Linux hat diese Idee etwas in den Hintergrund gedrängt.

Ein Kennzeichen der GNU Software, soweit sie UNIX Kommandos reimplementiert, sind Namen für die Kommandos, die so ähnlich klingen wie das Original, aber nicht identisch sind. So heißt das UNIX Kommando `more` in der GNU-Variante `less`.

6.1 Hiiiilfeeee!

Vor allem für Anfänger aber auch mit mehr Erfahrung kommt immer wieder der Moment, an dem man bei der Bedienung von UNIX nicht mehr weiter weiß, welches Kommando oder welche Option nun zum Ziel führt.

man: Das Online Manual auf jedem UNIX Rechne

Wenn man das passende Kommando kennt und wissen möchte, wie es verwendet wird, kann man das UNIX Online-Manual („*man-Pages*“) befragen. Das Kommando dazu heißt `man`. Das gesuchte Kommando gibt man als Parameter mit. Möchte man also beispielsweise wissen, wie das Online Manual bedient wird, gibt man „`man man`“ ein.

---

man gezielt die externe Variante benutzen, muss man einfach das Kommando mit dem Pfad angeben, also `/bin/echo`.

[11] http://www.fsf.org. Laut ihrem Gründer Richard Stallman soll man das „free“ bei freier Software eher im Sinne von freier Rede als von Freibier verstehen.

Der Aufbau der Manuals, die nun seitenweise angezeigt werden, ist standardisiert. Normalerweise enthalten Sie die Abschnitte

NAME	Enthält den Namen und eine Kurzbeschreibung des Kommandos
SYNTAX	Listet die Optionen und Parameter, die das Kommando kennt, auf
BESCHREIBUNG/ DESCRIPTION	Ausführliche Beschreibung, was das Kommando tut und wozu es eingesetzt werden kann
OPTIONEN/ OPTIONS	Beschreibt die einzelnen Optionen des Kommandos und ihre Wirkung
EINVIRONMENT	Zählt die Umgebungsvariablen auf, die das Kommando auswertet
SIEHE AUCH/ SEE ALSO	Zählt verwandte Kommandos auf
BUGS	Listet bekannte Schwächen des Kommandos auf

Man-Pages gibt es nicht nur für Kommandos, sondern auch für Bibliotheksfunktionen und Kernelaufrufe (*System Calls*), die man bei der C-Programmierung benötigt und für Dateiformate.

Um Ordnung in die Vielzahl der Seiten zu bringen, ist das Online Manual in Kapitel aufgeteilt

1	Benutzerommandos
2	Systemaufrufe
3	Bibliotheksfunktionen
4	Special files (dahinter verbergen sich die „Dateien“, über die zum Beispiel die Hardware angesteuert wird)
5	Dateiformate

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

6 Spiele (meistens leer)

7 Makropakete und Begriffe (zum Beispiel auch eine Seite über den Aufbau und die Verwendung von URLs.

8 System-Administrationskommandos

9 Kernel-Routinen (das ist eine Spezialität und kein UNIX Standard)

Manche Stichworte kommen mehrfach vor. Das Wort passwd kann sich sowohl auf ein Kommando (Kapitel 1) als auch auf das Format der Datei /etc/passwd (Kapitel 5) beziehen.

Um gezielt die eine oder andere Seite aufzurufen, kann man die Kapitelnummer vor den Suchbegriff setzen

```
bash > man 1 passwd
```

liefert also Informationen zur Verwendung des Kommandos passwd (dies ist der Standard), während

```
bash > man 5 passwd
```

über das Format der Datei /etc/passwd informiert.

Wenn man nicht weiß, wie das Kommando heißt, das man für einen bestimmten Zweck benötigt, kann man das man-Kommando mit der Option -k („*Keyword*") oder das Kommando apropos verwenden[12]

```
bash > man -k passwort
passwd               (1)  - Das Passwort des Benutzers ändern
putpwent             (3)  - schreibe einen Passwortdatei-Eintrag
shadow-utils        (rpm) - Dienstprogramme zum Verwalten von Shadow-
Passwortdateien und Accounts.
system-config-rootpassword (rpm) - Ein grafisches Interface zur Ände-
rung des Root-Passworts
```

[12] Das setzt voraus, dass eine entsprechende Datenbank von dem Programm /usr/sbin/mkwhatis angelegt wurde. Normalerweise wird dieses Programm im Rahmen der Systeminstallation ausgeführt. Falls nicht, kann man es jederzeit (als Benutzer *root*) ausführen. Das dauert aber ein Weilchen.

```
bash >
```

Neben dem vermutlich gesuchten passwd-Kommando findet man auch Verweise auf eine Bibliotheksfunktion, mit der man in einem C-Programm einen Passwortdatei-Eintrag schreiben kann, sowie zwei Hilfsprogramme des Linux Softwarepakete-Managers `rpm`. Das Kapitel rpm ist kein UNIX Standard und findet sich nur auf Systemen, die rpm verwenden (RedHat/Fedora, S.u.S.E und andere).

info: die verfeinerte GNU-Variante des Online-Manuals

Eine verfeinerte Variante des Online Manuals unter UNIX ist `info`. Dieses Kommando stammt aus der GNU Welt. Daher sind es vor allem die Programme aus diesem Umfeld, für die `info`-Seiten vorliegen. Gegenüber den man-Pages haben sie den Vorteil, Hyperlinks zu enthalten. Obwohl `info` ein textbasiertes System ist, kann man innerhalb der `info`-Seiten wie auf einer Webseite navigieren. Gibt es für ein Stichwort keine `info`-Seite, verwendet das Kommando die entsprechenden man-Pages.

```
bash > info bash
```

liefert beispiesweise

```
File: bash.info,  Node: Top,  Next: Introduction,  Prev: (dir),  Up: (dir)

Bash Features
*************

This text is a brief description of the features that are present in
the Bash shell (version 3.1-beta1, 5 September 2005)..

   This is Edition 3.1-beta1, last updated 5 September 2005, of `The
GNU Bash Reference Manual', for `Bash', Version 3.1-beta1.

   Bash contains features that appear in other popular shells, and some
features that only appear in Bash.  Some of the shells that Bash has
borrowed concepts from are the Bourne Shell (`sh'), the Korn Shell
(`ksh'), and the C-shell (`csh' and its successor, `tcsh'). The
following menu breaks the features up into categories based upon which
one of these other shells inspired the feature.

   This manual is meant as a brief introduction to features found in
Bash.  The Bash manual page should be used as the definitive reference
on shell behavior.

--zz-Info: (bash.info.gz)Top, 50 Zeilen --Top----*** Tags veraltet ***----------
Willkommen bei Info (Version 4.8). „?“ eingeben für Hilfe, „m“ für Me
```

Man kann sich nun mit den Cursortasten bewegen. Wörter, vor denen ein „*“ steht, sind Hyperlinks. Wenn der Cursor auf diesen Wörtern steht, kann man durch Drücken der Enter-Taste dem Link folgen. Im Zweifel hilft auch hier „`info info`“

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Wenn weder man noch info weiterhelfen, weil es zu einer Anwendung keine solchen Seiten gibt, kann man verschiedene Dinge ausprobieren, um an Informationen zur Bedienung zu kommen

- Das Kommando ohne weitere Parameter eingeben
- Das Kommando mit einer der Optionen `-h`, `-H`, `-?` oder `--help` ausführen

6.2 Verzeichnisse

Wie bei allen modernen Betriebssystemen sind auch die Dateien auf einem UNIX Rechner genauso wie bei einem Windows-PC in einem hierarchischen Dateisystem organisiert. Hierarchisch heißt, dass es Verzeichnisse gibt, die Dateien oder Unterverzeichnisse enthalten können. Die genaue Lage einer Datei wird durch die Angabe des Pfades, beginnend bei der Wurzel („*root*“) des Dateisystems, beschrieben. Es gibt allerdings einen wichtigen Unterschied zwischen den Dateisystemen der Microsoft-Betriebssysteme und dem UNIX Dateisystem: die von DOS und Windows her gewohnten Laufwerksbuchstaben findet man bei UNIX nicht. stattdessen sind alle Dateien und Verzeichnisse in einem einzigen Baum organisiert, dessen Wurzel mit einem „Slash“ („/“) bezeichnet wird. Die einzelnen Ebenen der Verzeichnisstruktur werden ebenfalls duch Slashes getrennt.

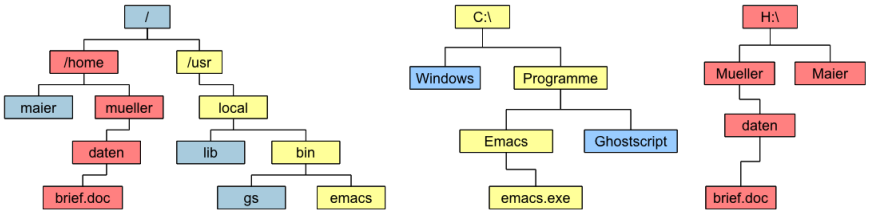


Abbildung 6.1: *Vergleich des Aufbaus der Dateisysteme unter UNIX (links) und MS Windows (rechts).*

In Abbildung 6.1 sind ist der Aufbau der Dateisysteme auf einem UNIX- und einem Windows-Rechner im Vergleich dargestellt Während das Windows Dateisystem zwei Laufwerke enthält (C:\ für das Betriebssystem und H:\ für die Benutzerdaten), sind die Verzeichnisse

im UNIX Dateisystem alle unter einer gemeinsamen Wurzel angeordnet[13].

Dem Pfad `/home/mueller/daten/brief.doc` unter UNIX entspräche ein Pfad wie `H:\Mueller\daten\brief.doc` unter MS Windows.

Beginnt eine Pfadangabe unter UNIX mit einem Slash, so bezeichnet sie die Lage eines Verzeichnisses oder einer Datei bezüglich der Wurzel des Dateisystems („*absoluter Pfad*“) . Eine Pfadangabe ohne führenden Slash bezeichnet die Lage eines Verzeichnisses oder einer Datei bezüglich des aktuellen Verzeichnisses („*relativer Pfad*“).

Spezielle Pfadnamen:

`/`	Wurzelverzeichnis
`.`	Aktuelles Verzeichnis
`..`	Übergeordnetes Verzeichnis
`~`	Heimatverzeichnis des aktuellen Benutzers
`~max`	Heimatverzeichnis des Benutzers `max`

pwd

Das Kommando pwd („*path working directory*“) zeigt den Pfad des aktuellen Verzeichnisses an

```
bash > pwd
/home/musterma
bash >
```

[13] In neueren Windows-Versionen gibt es ebenfalls die Möglichkeit Festplatten bzw. Partitionen in den Dateisystembaum einzubinden, statt ihnen einen eignenen Laufwerskbuchstaben zuzuordnen. Im Gegenzug kann auch das UNIX Dateisystem aus verschiedenen Partitionen aufgebaut sein, die dann, für den Benutzer zunächst unmerklich, zu einem einzigen Verzeichnisbaum zusammengefügt sind.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

ls

Das Kommando ls (*„list“*) zeigt den Inhalt (Dateien und Unterverzeichnisse) eines Verzeichnisses an

`ls texte` Zeigt den Inhalt des Unterverzeichnisses texte des aktuellen Verzeichnisses an

`ls -a` Zeigt den Inhalt des aktuellen Verzeichnisses an, wobei auch „versteckte“ Dateien (deren Name mit einem Punkt beginnt) angezeigt werden

`ls -l /etc` Zeigt den Inhalt des Verzeichnisses /etc in langer Form an. Dabei haben die einzelnen Felder der Ausgabe folgende Bedeutung

```
-rw-r--r-- 1 root root 1723 23. Sep 17:48 passv
| 1 |   2   |3| 4 | 5 | 6 |     7      |  8
```

1. Das erste Zeichen gibt an, um was es sich bei diesem Verzeichniseintrag handelt: „-“ bezeichnet beispielsweise eine normale Datei, ein „d“ bezeichnet ein Verzeichnis.
2. Diese neun Zeichen beschreiben die Zugriffsrechte auf dieses Objekt
3. Die Zahl der Verzeichnis-Einträge, die auf dieses Objekt zeigen („Link-Count“)
4. Der Eigentümer des Objektes
5. Die Benutzergruppe, der das Objekt zugeordnet ist
6. Die Größe des Objektes in Byte
7. Datum und Uhrzeit der letzen Änderung
8. Der Name des Objektes

`ls -lh ..` Zeigt den Inhalt des aktuellen Verzeichnis übergeordneten Verzeichnisses in langer Form an, wobei die Dateigrößen in kB, MB oder GB („*human readable*“) angezeigt werden

ls -R / Zeige den Inhalt des Wurzelverzeichnisses und rekursiv den Inhalt aller Unterverzeichnisse an

Übungen:

Versuchen Sie, aus der man-Page von `ls` herauszubekommen

1. Wie Sie bei der kurzen Form des Listings ausführbare Dateien und Verzeichnisse besonders kennzeichnen können (Stichwort: „Typisierung“ – falls Sie die deutschen Manpages verwenden)

2. wie Sie sich die Dateien im Verzeichnis `/etc` sortiert nach dem Datum der letzten Änderung anzeigen lassen können, beginnend mit der ältesten bis zur neuesten

cd

Mit dem Kommando cd („*change directory*“) kann man sich durch den Verzeichnisbaum bewegen. Ohne weitere Parameter aufgerufen, wechselt cd in das Heimatverzeichnis des Benutzers[14]. Die wichtigsten der zahllosen Varianten von cd sind

`cd ..` Wechselt in das übergeordnete Verzeichnis

```
bash > pwd
/usr/local/bin
bash > cd ..
bash > pwd
/usr/local
bash >
```

[14] Dieses Verhalten steht im Kontrast zu MS DOS/Windows, wo das Kommando *cd* das aktuelle Verzeichnis anzeigt, so wie das UNIX-Kommando *pwd.*

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

`cd /usr/lib` Wechselt in das Verzeichnis /usr/lib (*absoluter Pfad*)

```
bash > pwd
/home/musterma
bash > cd /usr/lib
bash > pwd
/usr/lib
bash >
```

`cd X11` Wechselt ausgehend vom aktuellen Verzeichnis in das Unterverzeichnis X11 (*relativer Pfad*)

```
bash > pwd
/usr/lib
bash > cd X11
bash > pwd
/usr/lib/X11
bash >
```

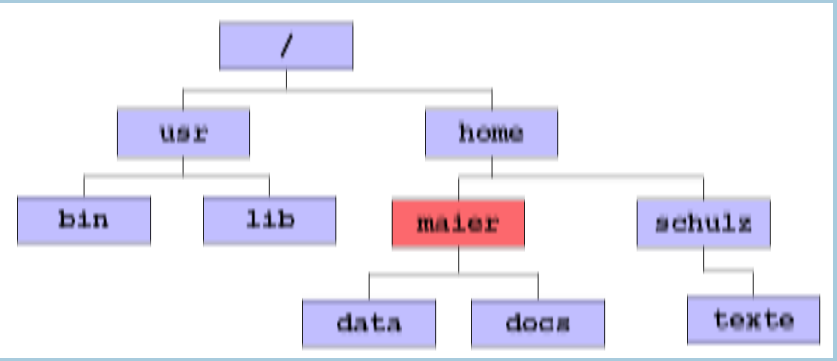
`cd -` Wechselt in das Verzeichnis, von dem aus der letzte cd-Aufruf erfolgte

```
bash > pwd
/home/musterma
bash > cd /etc
bash > pwd
/etc
bash > cd -
bash > pwd
/home/musterma
```

`cd ../maier` Wechselt in das Unterverzeichnis maier des übergeordneten Verzeichnisses

```
bash > pwd
/home/musterma
bash > cd ../maier
bash > pwd
/home/maier
bash >
```

Übungen:



Sie befinden sich im Verzeichis `/home/maier` und führen folgende cd-Kommandos aus:

- `cd docs`
- `cd /usr/lib`
- `cd -`
- `cd ../../schulz`
- `cd -`

In welchem Verzeichnis befinden Sie sich nun?

mkdir

Mit dem Kommando mkdir („*make directory*“) erzeugt man ein neues Verzeichnis

```
bash > mkdir src
```

erzeugt ein Unterverzeichnis src im aktuellen Verzeichnis (*relativer Pfad*).

```
bash > mkdir /usr/local/teTeX
```

erzeugt ein Unterverzeichnis `teTeX` im Verzeichnis `/usr/local`, sofern letzteres existiert (*absoluter Pfad*).

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Will man eine ganze Hierarchie von Verzeichnissen erzeugen, so kann man die Option -p („*path*“) verwenden

```
bash > mkdir -p data/alt data/neu
```

erzeugt (wenn es nicht schon existiert) das Verzeichnis `data` und darin die Unterverzeichnisse `alt` und neu.

rmdir

Mit dem Kommando `rmdir` können Sie ein Verzeichnis löschen, allerdings nur, wenn es leer ist.

```
bash > rmdir data/neu
bash > rmdir data
rmdir: data: Das Verzeichnis ist nicht leer
bash >
```

Das Verzeichnis data enthält noch das eben angelegte Unterverzeichnis alt. Um das Verzeichnis data löschen zu können, muss man entweder zuerst das Verzeichnis data/alt löschen, oder man verwendet wiederum die Option -p („path“)

```
bash > rmdir -p data/alt
bash >
```

Damit wird der gesamte aufgeführte Pfad gelöscht. enthält allerdings eins der beteiligten Verzeichnisse noch Dateien oder andere Unterverzeichnisse, so stoppt der Löschvorgang an dieser Stelle.

Um ganze Verzeichnisbäume mit Dateien zu löschen, muss man das Kommando `rm` (s. Seite 34) verwenden.

Die Antworten finden Sie auf Seite 96. Aber versuchen Sie in jedem Fall erst einmal, selbst auf die Lösung zu kommen und schauen Sie im Zweifelsfall noch einmal im Text nach. Durch „Schummeln“ schaden Sie niemanden – außer sich selbst.

Fragen zur Sebstkontrolle

1. Woran erkennt man eine absolute Pfadangabe in UNIX

2. Was bewirkt das Kommando `cd -` ?

3. Mit welchem UNIX-Kommando können sie in Ihrem aktuellen, leeren Verzeichnis den Verzeichnisbaum `develop/java/src` erzeugen?

4. Wann funktioniert das Kommando `rmdir` **nicht**?

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

6.3 Dateien

touch

Das touch Kommando tut genau das, was der Name nahe legt – es berührt eine Datei oder ein Verzeichnis. Das heißt, Datum und Uhrzeit der letzten Änderung werden auf die aktuelle Systemzeit gesetzt. Existiert die Datei nicht, wird sie angelegt.

```
bash > ls -l textdatei
-rw-rw-r-- 1 musterma users    4 22. Sep 14:31 textdatei
bash > touch textdatei
bash > ls -l textdatei
-rw-rw-r-- 1 musterma users    4 29. Sep 16:10 textdatei
bash >
```

cp

Kopie: Aus einer Datei werden zwei unabhängige Dateien

Das Kommando cp („*copy*“) erzeugt eine Kopie einer oder mehrerer Dateien.

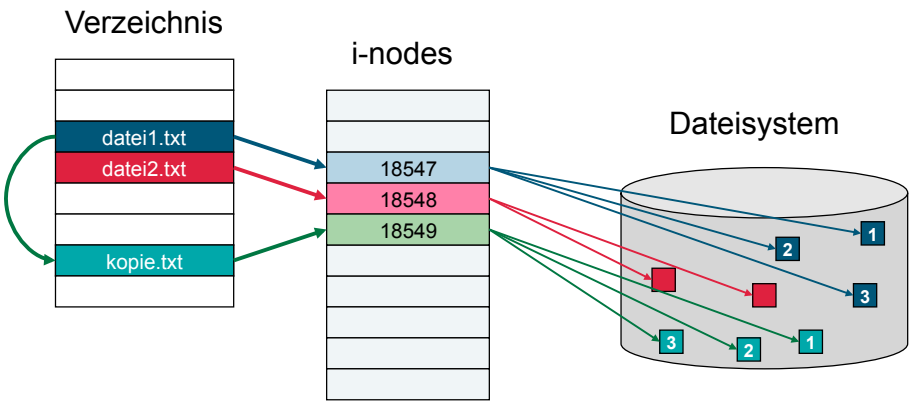


Abbildung 6.2: *Schematische Funktion des cp Kommandos*

Dazu wird zunächst ein neuer Verzeichniseintrag mit dem neuen Namen erzeugt. Die Detailinformationen über die Datei (Eigentümer, Zugriffsrechte, ...) stehen in einer Datenstruktur, die als *Index-node* („*i-node*“) bezeichnet wird. Für die Kopie wird ein neuer i-node angelegt. Schließlich werden noch die Daten kopiert. Das heißt, Original und Kopie belegen zusammen doppelt so viel Plattenplatz wie die Originaldatei alleine.

Original und Kopie sind zwei völlig verschiedene Dateien. Änderungen an einer der beiden Dateien sind bei der anderen Datei nicht sichtbar.

Übung:

- Erzeugen Sie eine Textdatei `original.txt` (zum Beispiel mit „`kedit original.txt`“) und schreiben Sie irgend einen Text in diese Datei.
- Kopieren Sie die Datei: `cp original.txt kopie.txt`
- Vergewissern Sie sich, dass in der kopierten Datei der gleiche Text steht, wie in der ursprünglichen Datei
- Ändern Sie die Originaldatei
- Überprüfen Sie, dass die Kopie unverändert geblieben ist

ln

Hardlink: Ein zusätzlicher Verzeichniseintrag zeigt auf den i-node einer bereits existierenden Datei. Alle Verzeichniseinträge, die auf einen i-node zeigen, sind völlig gleichberechtigt

Das Kommando `ln` („*link*“) erzeugt einen weiteren Verzeichniseintrag zu einer existierenden Datei. Dabei gibt es zwei Varianten:

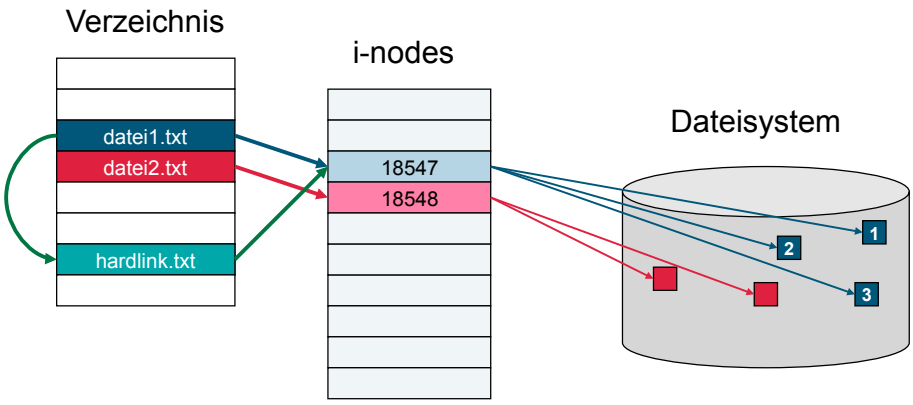


Abbildung 6.3: *Schematische Funktion des ln-Kommandos bei der Erstellung eines Hardlinks*

Bei einem **Hardlink** wird ein zusätzlicher Verzeichniseintrag erzeugt, der auf den i-node der ursprünglichen Datei zeigt. Die Daten werden nicht kopiert. Das heißt, es sieht so aus, als ob zwei Dateien da wären, obwohl die Daten nur einmal auf der Festplatte gespeichert sind.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Ändert man eine der beiden Dateien, so ist diese Änderung auch in der anderen Datei sichtbar (in Wirklichkeit gibt es ja nur eine Datei). Löscht man die eine „Datei“, so ist die andere unverändert vorhanden[15]. Im „langen Listing“ einer Datei kann man sehen, wie viele Verzeichniseinträge auf den entsprechenden i-node zeigen (s. Seite 23).

Da die i-nodes zu einer Datei Teil des Dateisystems einer Festplattenpartition sind, müssen auch alle Verzeichniseinträge, die auf einen i-node zeigen, in derselben Partition liegen. Hardlinks funktionieren deshalb nur innerhalb einer Partition. Hardlinks funktionieren auch nur für normale Dateien. Bei Verzeichnissen sind sie nicht zulässig[16].

```
bash > ls -l orignal.txt kopie.txt
-rw-rw-r-- 1 musterma users    27 29. Sep 17:00 kopie.txt
-rw-rw-r-- 1 musterma users    39 29. Sep 17:15 original.txt
bash >
```

Bei Original und Kopie ist der Link-Count jeweils 1, Größe und Zeitstempel unterscheiden sich. Mit der Option –i zeigt ls am Zeilenanfang die Nummer des i-nodes, auf den ein Verzeichniseintrag zeigt

```
bash > ls -li orignal.txt kopie.txt
391900 -rw-rw-r-- 1 musterma users 27 29. Sep 17:00 kopie.txt
391913 -rw-rw-r-- 1 musterma users 39 29. Sep 17:15 orignal.tx
bash >
```

Erzeugt man einen Hardlink, ist der Link-Count bei beiden Verzeichniseinträgen 2, Größe, Zeitstempel und i-node Nummer sind gleich

```
bash > ln original.txt hardlink.txt
bash > ls -li original.txt hardlink.txt
391913 -rw-rw-r-- 2 musterma users 39 29. Sep 17:15 hardlink.t
391913 -rw-rw-r-- 2 musterma users 39 29. Sep 17:15 orignal.tx
bash >
```

---

[15] Der Systemaufruf, der beim Löschen von Dateien und Verzeichnissen zum Einsatz kommt, heißt *unlink*. Das heißt, es wird zunächst nur ein Verzeichniseintrag („*link*“) gelöscht. Erst wenn der letzte Link entfernt ist, werden auch die Daten gelöscht.

[16] Trotzdem haben Verzeichnisse immer einen Linkcount > 1. Denn es gibt in jedem Fall zwei Verzeichniseinträge zu jedem Verzeichnis. Den Eintrag im übergeordneten Verzeichnis und den Entrag „.“ im Verzeichnis selbst. Enthält das Verzeichnis Unterverzeichnisse, kommt in jedem Unterverzeichnis ein Verzeichniseintrag „..“ dazu. Aber diese Links können nur automatisch vom Betriebssystem erstellt werden. Als Benutzer(in) ist man hier chancenlos.

Übung:

- Verändern Sie die neu erzeugte Datei `hardlink.txt`
- Überprüfen Sie, dass die Originaldatei auch geändert ist
- Vergleichen Sie i-node Nummer, Größe und Zeitstempel von Original, Kopie und Hardlink
- Löschen Sie die Datei `original.txt` und vergewissern Sie sich, dass die Datei `hardlink.txt` unverändert vorhanden ist (bis auf den Link-Count, der jetzt auf 1 steht).

Symlink: Ein zusätzlicher Verzeichniseintrag, der den Pfad der Datei enthält, auf den der Link zeigt.

Bei einem **symbolischen Link** („*Symlink*“) wird ebenfalls ein zusätzlicher Verzeichniseintrag erzeugt, der aber im Gegensatz zum Hardlink nicht auf einen i-node sondern auf einen anderen Verzeichniseintrag verweist. Symlinks entsprechen den Verknüpfungen von MS Windows.

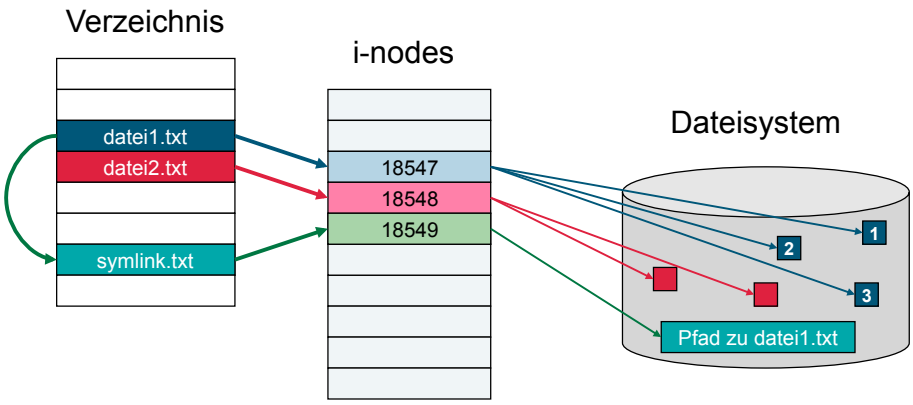


Abbildung 6.4: *Schematische Darstellung der Funktion des Kommandos ln bei der Erzeugung eines symbolischen Links*

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Übung:

- Erzeugen Sie zunächst die Datei original.txt wieder, indem Sie einen Hardlink von hardlink.txt auf original.txt erzeugen.
- Erzeugen Sie dann mit dem Kommando
  ln -s original.txt symlink.txt
  einen symbolischen Link
- Lassen Sie sich Original und Symlink in langer Form anzeigen

```
bash > ls -l original.txt symlink.txt
-rw-rw-r-- 2 musterma users    39 29. Sep 17:15 original.txt
lrwxrwxrwx 1 musterma users    12 29. Sep 17:42 symlink.txt ->
original.txt
bash >
```

Der Verzeichniseintrag des Symlinks unterscheidet sich deutlich von dem der Originaldatei

- Das erste Zeichen in der Zeile ist ein „l“ statt eines Bindestrichs. Das ist das Zeichen, dass es sich um einem symbolischen Link handelt
- Der Linkcount ist eins
- Die Größe beträgt wenige Bytes. Dieser Effekt ist natürlich deutlicher, wenn man einen Symlink einer großen Datei erzeugt
- Im Namen steht der Pfad des Verzeichniseintrags, auf den der Symlink zeigt

Wenn man die i-node-Nummern von Original und Symlink vergleicht, sieht man, dass der Symlink eine eigene i-node Nummer hat[17].

Symlinks haben zwei große Vorteile gegenüber Hardlinks. Zum einen kann man sie auch auf Verzeichnisse anwenden. Und zum anderen können Sie auch auf Pfade zeigen, die auf anderen Datenträgern oder sogar auf anderen Computern liegen. Sie haben aber auch einen Nachteil

---

[17] Eigentlich ist der Symlink eine kleine Datei, die nur den Pfad enthält, auf den der symlink zeigt. Diese Datei hat einen natürlich eigenen i-node und verbraucht einen einzigen Plattenblock im Dateisystem.

Übung:

- ❑ Löschen Sie die Datei original.txt
- ❑ Versuchen Sie, die Datei Symlink zu öffnen

```
bash > cat symlink.txt
cat: symlink.txt: Datei oder Verzeichnis nicht gefunden[18]
bash >
```

Der Symlink kann nicht überprüfen, ob sich das Ziel des Links verändert hat oder ob dieses Ziel überhaupt noch existiert. Deshalb können solche „toten Links“ entstehen, wenn die Zielobjekte der Links nicht mehr vorhanden oder umbenannt wurden

rm

Achtung: Das rm-Kommando ist schnell und mächtig. Bei seiner Verwendung ist immer allergrößte Konzentration gefragt!

Das Kommando `rm` („*remove*“) hat schon viele UNIX Benutzer und vor allem Administrator(inn)en ins Unglück gestürzt. Man sollte sich angewöhnen, dieses Kommando nur mit allergrößter Vorsicht zu verwenden!

`rm blubb`	Löscht die Datei blubb im aktuellen Verzeichnis. Verzeichnisse können auf diese Weise nicht gelöscht werden.
`rm -r bla`	Löscht rekursiv das Verzeichnis bla und alle darin enthaltenen Dateien und Verzeichnisse. **Vorsicht vor Risiken und Nebenwirkungen!!!**
`rm -i *.o`	Löscht alle Dateien, deren Name mit „.o“ endet. Bei jeder Datei wird der Benutzer gefragt, ob sie wirklich gelöscht werden soll.
`rm -f *`	Löscht alle Dateien im aktuellen Verzeichnis ohne Rückfrage

[18] Achtung: Die Fehlermeldung könnte suggerieren, dass es symlink.txt ist, das nicht gefunden werden konnte. In Wirklichkeit ist diese Datei aber der Ort, an dem der Fehler aufgetreten ist.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Streng genommen entfernt das rm-Kommando lediglich den angegebenen Verzeichniseintrag. Erst wenn der letzte Verzeichniseintrag entfernt wurde, gibt das Betriebssystem den i-node und den Plattenplatz, den die Datei belegt hat, frei. Möchte man vertrauliche Daten zuverlässig löschen, reicht das rm-Kommando daher nicht aus. Spezielle Programme überschreiben die betroffenen Plattenbereiche mehrfach mit zufälligen Bitmustern, um eine Wiederherstellung der gelöschten Dateien zu verhindern.

mv

Das Kommando mv („move“) verschiebt eine Datei oder ein Verzeichnis. Das Kommando lässt sich auch dazu verwenden, eine Datei umzubenennen.

mv: Ersetzt den Verzeichniseintrag zu einer Datei durch einen neuen

Innerhalb einer Plattenpartition wird lediglich der Verzeichniseintrag durch einen neuen ersetzt. I-node und Daten bleiben unberührt. Wird eine Datei oder ein Verzeichnis von einem Datenträger zu einem anderen verschoben, werden die Daten kopiert und danach am alten Ort gelöscht.

```
bash > ls -li quelle
391897 -rw-rw-r-- 1 musterma users    30 29. Sep 18:45 quelle
bash > mv quelle sub/ziel
bash > cd sub
bash > ls -li ziel
391897 -rw-rw-r-- 1 musterma users    30 29. Sep 18:45 ziel
bash >
```

Durch das Verschieben wurden weder die Größe noch der Zeitstempel oder die i-Node Nummer geändert.

cat

Eigentlich ist das Kommando cat („*concatenate*“ – „*verketten*“) dazu gedacht, mehrere Dateien zu einer langen Datei zusammen zu fassen.

```
bash > cat datei1 datei2 > ziel[19]
```

[19] Das Zeichen „>“ weist die Shell an, die Ausgaben, die das Kommando cat sonst auf dem Bildschirm machen würde, in die Datei zu schreiben, deren Name nach dem „>“ Zeichen folgt.

Oft wird das Kommando allerdings dazu verwendet, den Inhalt einer Datei auf dem Bildschrim auszugeben

```
bash > cat /etc/hosts
```

more/less

Das Kommando `more` dient dazu, eine Textdatei auf dem Bildschirm auszugeben, wobei das Programm eine Pause einlegt, wenn der Bildschirm gefüllt ist. In der linken unteren Ecke signalisiert das Wort „more“, dass noch weitere Zeilen anzuzeigen sind. Drückt der Benutzer die Leertaste, wird eine Seite weiter geblättert.

Die Open-Source Implementierung des more-Kommados kann mehr als das Original und hat deshalb einen eigenen Namen: `less`. Auf Linux-Rechnern sind beide Kommandos verfügbar. Beide Kommandos haben eine Reihe von nützlichen Optionen

`more -10 /tmp/protocol`	Gibt die Datei `/tmp/protocol` in Portionen von 10 Zeilen aus
`more -s langedatei`	(„*squeeze*“): Gibt die Datei `langedatei` seitenweise aus, wobei mehrere aufeinander folgende Leerzeilen durch eine einzige ersetzt werden
`more +254 /var/log/syslog`	Zeigt die Datei `/var/log/syslog` ab Zeile 254 an
`more +/failure /var/log/syslog`	Zeigt die Datei `/var/log/syslog` an. Beginne zwei Zeilen bevor das Wort „*failure*“ das erste Mal vorkommt

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Während der Ausführung von more/less gibt es eine Reihe wichtiger Tastaturkommandos

`[Leertaste]`	Blättert eine Seite vorwärts
`[Enter]`	Blättert eine Zeile vorwärts
`b`	Blättert eine Seite zurück
`g`	Springt zum Anfang der Datei (nur bei less)
`G`	Springt zum Ende der Datei (nur bei less)
`/blubb`	Sucht vorwärts nach dem nächsten Auftreten der Zeichenkette „*blubb*“
`?blubb`	Sucht rückwärts nach dem nächsten Auftreten der Zeichenkette „*blubb*“ (nur `less`)
`n`	Wiederhole die letzte Suche
`q` oder `Q`	Beenden von `more/less`

head

Das Kommando head gibt den Anfang einer oder mehrerer Dateien aus. Ohne weitere Optionen werden die ersten 10 Zeilen ausgegeben.

`head laber.txt`	Gibt die ersten zehn Zeilen der Datei laber.txt aus
`head -3 bla.c blubb.c` *oder* `head -n 3 ...` *oder* `head --lines 3 ...`	Gibt jeweils die ersten drei Zeilen der Dateien `bla.c` und `blubb.c` aus
`head -c 10 out.dat` *oder* `head --bytes=10 ...`	Gibt die ersten zehn Zeichen der Datei `out.dat` aus

tail

Das Kommando `tail` gibt das Ende einer oder mehrerer Dateien aus. Ohne weitere Optionen werden die letzten 10 Zeilen ausgegeben.

`tail laber.txt`	Gibt die letzten zehn Zeilen der

	Datei laber.txt aus
`tail -3 bla.c blubb.c` *oder* `tail -n 3 ...` *oder* `tail --lines 3 ...`	Gibt jeweils die letzten drei Zeilen der Dateien `bla.c` und `blubb.c` aus
`tail -c 10 out.dat` *oder* `tail --bytes=10 ...`	Gibt die letzten zehn Zeichen der Datei `out.dat` aus
`tail -f /var/log/messages`	(„*filter*") Gibt die letzten zehn Zeilen der Datei `/var/log/messages` aus und wartet dann auf weitere Zeilen, die an die Datei angehängt werden und die dann ausgegeben werden. Auf diese Weise kann man zum Beispiel Protokolldateien „live" im Auge behalten

od

Das Kommando od („*Octal dump*") dient dazu, beliebige Dateien in lesbarer Form auszugeben. Damit lassen sich nicht-druckende Zeichen in Textdateien identifizieren oder der Inhalt von Nicht-Textdateien in Augenschein nehmen. Über Optionen lässt sich das Ausgabeformat bestimmen

-a Stellt nicht-druckende Zeichen in ASCII Notation dar (zum Beispiel „sp" für ein Leerzeichen)

-c Stellt nicht-druckende Zeichen durch C-artige Escape-Sequenzen dar (zum Beispiel \n am Zeilenende)

-x Stellt jeweils 16 Bit als vierstellige Hexadzimalzahl dar

... Und viele Möglichkeiten mehr

```
bash > cat erster_text
Anfang          und Ende

bash > cat zweiter_text
Anfang          und Ende
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

```
bash > od -c erster_text
0000000   A   n   f   a   n   g  \t  \t   u   n   d       E   n   d
e
0000020  \n  \n
0000022
bash > od -c zweiter_text
0000000   A   n   f   a   n   g
\t
0000020   u   n   d       E   n   d   e  \n  \n
0000032
bash >
```

Für das cat-Kommando sind die beiden Dateien erster_text und zweiter_text nicht zu unterscheiden. Erst mit od erkennt man, dass in der ersten Datei zwei Tabulator-Zeichen enthalten sind, während in der zweiten Datei nur Leerzeichen enthalten sind.

file

Das file-Kommando analysiert den Anfang einer Datei und bestimmt den Typ der Datei. Das funktioniert auch, wenn die Dateinamen-Erweiterung fehlt oder falsch ist

```
bash > file flowers.jpg
flowers.jpg: JPEG image data, JFIF standard 1.01
bash >
```

which

Bei Programmen, die im Suchpfad der Shell liegen, liefert das Kommando which den Pfad zu der zugehörigen Datei.

```
bash > which kedit
/usr/bin/kedit
bash >
```

find

Mit dem find-Kommando kann man Dateien in einem Dateisystem anhand ihres Namens, ihres Alters oder anderer Kriterien finden. Um beispielsweise alle JPEG-Bilder in einem bestimmten Verzeichnis zu finden, benutzt man den Befehl

```
bash > find /usr/share/pics -name \*.jpg
```

An erster Stelle in der Liste der Parameter steht immer der Pfad, unterhalb dessen nach Dateien gesucht werden soll. Danach kommen ein oder mehrere Parameter, dir mit einem „-„ beginnen. In diesem Beispiel wird nach Dateien gesucht, deren Name mit „.jpg“ endet. Der Backslash verhindert, dass der Stern von der Shell expandiert wird.

Das find-Kommando kann auch nach Dateitypen (Datei, Verzeichnis, Symbolischer Link, ...) suchen Im folgenden Beispiel werden alle symbolischen Links unterhalb von /etc gesucht.

```
bash > find /etc -type l
```

Eine wichtige Anwendung von find ist auch die Suche nach Dateien, die zu bestimmten Zeiten geöffnet waren oder verändert wurden. Die Manpage von find gibt Auskunft über die verschiedenen Möglichkeiten.

Übungsaufgaben:

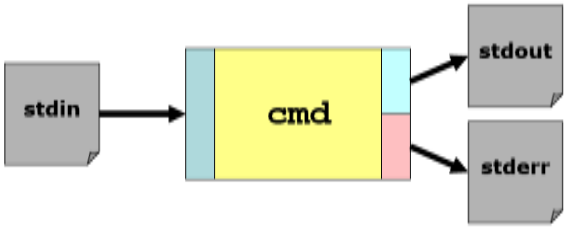
- Suche alle Dateien, die vor weniger als drei Tagen geändert wurden
- Suche alle Dateien, die in den letzten 10 Minuten geöffnet waren
- Suche alle Dateien, die nach der Datei **MARKER** erzeugt wurden

6.4 Umlenkung von Ein- und Ausgabe

Eines der Grundprinzipien von UNIX lautet: „Alles ist Datei“. Die Kommunikation zwischen den Programmen oder die Ansteuerung von Hardware funktionieren genau so wie das Lesen von oder Schreiben in Dateien.

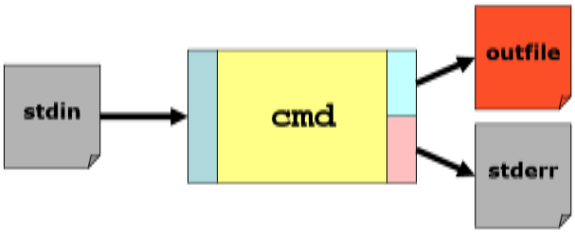
Jedes Programm, das auf einem UNIX-Rechner läuft, hat standardmäßig drei geöffnete Datei-Deskriptoren

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen



- **stdin**: Die Standardeingabe. Über diesen Kanal erhält das Programm normalerweise Daten von der Konsole (Dateideskriptor 0)
- **stdout**: Die Standard-Ausgabe. Über diesen Kanal schickt das Programm seine normalen Ausgaben an die Konsole (Dateideskriptor 1)
- **stderr**: Die Standard-Fehlerausgabe. Über diesen Kanal schickt das Programm seine Fehlermeldungen an die Konsole (Dateideskriptor 2)

Jeder dieser drei Kanäle kann von der Konsole getrennt und in eine Datei oder einen anderen Datenkanal umgelenkt werden.




```
bash > cmd > outfile
```
[20]

Die normalen Ausgaben des Kommandos *cmd* werden in die Datei `outfile` umgelenkt. Wenn die Datei bereits existiert, wird sie überschrieben. Möchte man die Ausgaben von cmd an die Datei outfile anhängen, muss man den Operator „>>“ verwenden

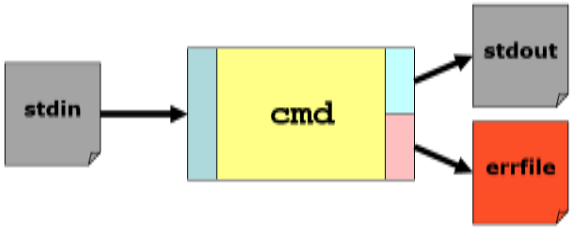
```
bash > cmd >> outfile
```
[21]

[20] cmd steht für irgend ein UNIX Kommando

[21] cmd steht für irgend ein UNIX Kommando

Die Ausgaben über den Standard-Fehlerausgabe Kanal werden aber in beiden Fällen auf der Konsole ausgegeben.

Aber auch diese Ausgaben lassen sich in eine Datei umlenken

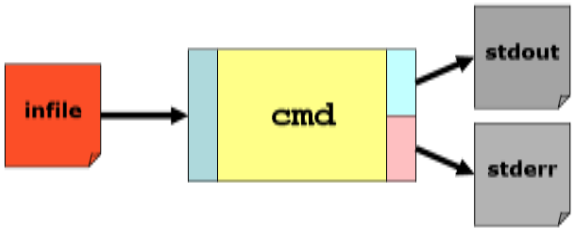



```
bash > cmd 2> errfile[22]
```

Und wieder gilt, dass eine existierende Datei errfile überschrieben wird. Zum Anhängen der Fehlermeldungen an eine existierende Datei gibt es aber wieder den Operator „2>>“

```
bash > cmd 2>> errfile
```

Schließlich lassen sich die Eingaben über die Standardeingabe statt von der Konsole aus einer Datei beziehen

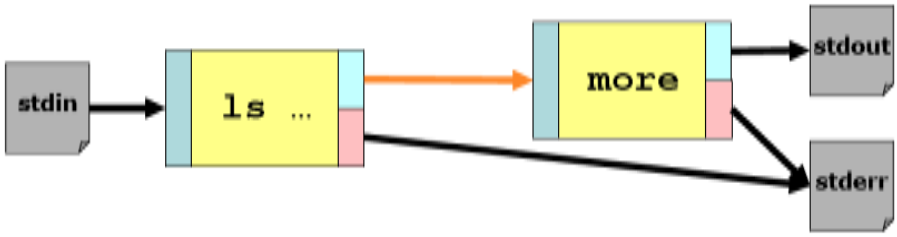



```
bash > cmd < infile
```

[22] Die 2 steht für den Dateideskriptor, mit dem der stderr-Kanal verbunden ist. Streng genommen müsste man den stdout-Kanal mit „1>“ umlenken. Aber da stdout der Standard für die Ausgabe ist, darf man die 1 weglassen. Prinzipiell kann man auch weitere Dateideskriptoren verwenden („3>“, „4>“, ...). In der Regel werden die Kommandos in diese Kanäle aber nichts schreiben.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Die UNIX-Shell bietet auch die Möglichkeit, die Ausgabe eines Kommandos als Eingabe eines zweiten Kommandos zu verwenden.




```
bash > ls -ltr /usr/bin | more
```

Das Verzeichnis `/usr/bin` kann über 2000 Dateien enthalten. Insofern ist eine seitenweise Untergliederung des Listings sinnvoll. Durch das „Pipe"-Zeichen („|") wird der stdout-Kanal des `ls`-Kommandos mit dem stdin-Kanal des `more`-Kommandos verknüpft.

Problematisch wird es allerdings, wenn bei dem ersten Kommando viele Fehler auftreten. Der stderr-Kanal von ls geht ungebremst auf die Konsole und zerstört auf diese Weise die seitenweise Aufbereitung der Ausgabe. Man kann dann beispielsweise nicht mehr rekonstruieren, an welcher Stelle der Programmausführung ein Fehler aufgetreten ist, weil stdout und stderr nicht mehr synchron sind.

Die Lösung dieses Problems besteht darin, den stderr-Kanal von ls in den stdout-Kanal umzulenken. Das funktioniert im Prinzip genauso wie die Umlenkung in eine Datei. Es muss jetzt nur die „Adresse" des Dateideskriptors des stdout-Kanals als Ziel der Umlenkung angegeben werden. Diese Adresse wird in der aus der C-Programmierung von Zeigern bekannten Schreibweise angegeben: „&1".

Damit ist der Operator für die Umlenkung von stderr nach stdout „2>&1" (lies: Dateideskriptor 2 wird mit der Adresse von Dateideskriptor 1 gefüllt

```
bash > ls -ltr /usr/bin 2>&1 | more
```

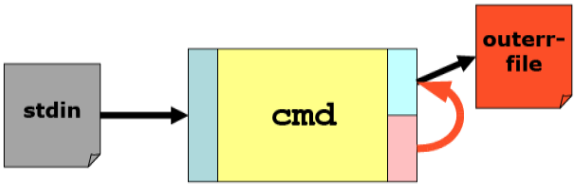
Wenn man mit vielen Fehlermeldungen rechnet und weiß, dass man sie ignorieren kann, besteht alternativ die Möglichkeit, den stderr-

/dev/null ist eine Spezialdatei, in die man beliebige Daten schreiben kann, die dort spurlos verschwinden.

Kanal mit dem „elektronischen Müllschlucker“ von UNIX `/dev/null` zu verbinden. In diese Spezialdatei kann man schreiben, was man will – es kommt niemals wieder.

```
bash > ls -ltr /usr/bin 2> /dev/null | more
```

Möchte man stdout- und stderr-Kanal in eine gemeinsame Datei umlenken, muss man sorgfältig die Reihenfolge der Umlenkungen beachten




```
bash > cmd > outerrfile 2>&1
```

Die Shell wertet die Kommandozeile von links nach rechts aus. Zunächst wird daher der stdout-Kanal auf die Datei `outerrfile` umgelenkt und dann der stderr-Kanal mit der Adresse des stdout-Kanals verbunden (in diesem Fall `outerrfile`).

Würde man die Reihenfolge der Umlenkungen vertauschen, würde zunächst der stderr-Kanal mit der Adresse des stdout-Kanals (also der Konsole) verbunden und dann erst der stdout-Kanal in die Datei `outerrfile` umgelenkt. Das bedeutet, die Fehlermeldungen landen weiterhin auf dem Bildschirm.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Fragen zur Sebstkontrolle

1. Erklären Sie in Stichworten den Unterschied zwischen einer Kopie, einem Hardlink und einem Symlink

2. Wenn man mit dem Kommando mv ein Verzeichnis mit tausenden von Dateien verschiebt, geht das ganz schnell, wenn man es innerhalb einer Partition verschiebt, aber viel langsamer, wenn man es von einer Partition auf die andere verschiebt. Warum?

3. Wie kann man sich die aktuellste Datei in einem Verzeichnis ausgeben lassen?

4. Wie kann man die vorletzte Zeile der Textdatei `brief.txt` anzeigen lassen?

5. Was bewirkt die UNIX Kommandozeile „`cmd 3>&2 2>&1 1>&3`“?

Die Antworten finden Sie auf Seite 96. Aber versuchen Sie in jedem Fall erst einmal, selbst auf die Lösung zu kommen und schauen Sie im Zweifelsfall noch einmal im Text nach. Durch „Schummeln“ schaden Sie niemanden – außer sich selbst.

6.5 Dateisysteme

mount

Bevor man einen Datenträger auf einem UNIX System nutzen kann, muss man die darauf gespeicherten Daten und Verzeichnisse in den globalen Dateisystem-Baum des UNIX-Systems einbinden. Dabei ist es egal. ob es sich um eine Diskette, CD, Festplatte oder einen Fileserver handelt. Diesen Vorgang nennt man „*mounten*“ (von englisch „*to mount*“ = anbringen, befestigen).

In der Regel kann der/die normale Benutzer(in) keine Datenträger mounten, da hierzu administrative Zugriffsrechte notwendig sind. In früheren Versionen von Linux brauchte man das `mount`-Kommando, um CDs, DVDs oder Floppy-Disks (die Älteren werden sich erinnern ...) unter Linux lesen zu können. Bei neueren Linux-Versionen übernimmt ein so genannter Automounter diese Aufgabe automatisch beim Einlegen des Datenträgers. Dieses Programm bindet einen Datenträger ein, sobald auf den entsprechenden Mountpunkt zugegriffen wird. Bei vielen neueren Linux-Distributionen ist der Automounter konfiguriert, so dass sich diese Systeme bei der Handhabung von Wechseldatenträgern fast wie Windows-Systeme verhalten[23]. Wichtig ist aber in jedem Fall, die Einbindung des Datenträgers zu lösen, bevor man ihn physikalisch entfernt.

Für Systemadministrator(inn)en ist das mount-Kommando aber immer noch wichtig. Um z.B. eine CD/DVD „von Hand“ zu mounten, wird die Kommandozeile

```
bash > mount /dev/dvd /media/dvd
```

verwendet. Das Verzeichnis `/media` heißt „Mountpunkt“ und muss vorher schon existieren.

Bevor man einen Datenträger aus einem UNIX-System entfernt, muss man unbedingt die Einbindung in das UNIX-Dateisystem lösen. An-

---

[23] Seit 2011 nutzen die meisten Linux-Distributionen das Programm `systemd`, um die verschiedenen Teile des Computers zu steuern und zu koordinieren. Dieses Programm übernimmt auch die Funktion des Automounters.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

dernfalls riskiert man den Verlust der darauf gespeicherten Daten. Das geschieht mit dem Kommando

```
bash > unmount /media/dvd
```

df

Das Kommando df („*disk free*“) zeigt an, welche Dateisysteme gemountet sind, wie groß diese sind und wie viel Platz noch frei ist.

```
bash > df
Dateisystem          1K-Blöcke   Benutzt Verfügbar Ben% Einge-
hängt auf
/dev/mapper/VolGroup00-LogVol00
                       5427652   4000064   1147424  78% /
/dev/hda1               101086      9854     86013  11% /boot
tmpfs                   128044         0    128044   0%
/dev/shm
bash >
```

In der ersten Spalte sind wieder die Spezialdateien aufgelistet, in der zweiten Spalte die Größe der Dateisysteme (in kB). Die nächsten beiden Spalten geben an, wie viel Plattenplatz belegt bzw. frei ist. Daraus wird ein Belegungsgrad berechnet. In der letzten Spalte ist der Mountpunkt aufgelistet.

Um die bei modernen Festplatten doch unübersichtlichen Angaben in kB zu vermeiden, kann man die Option –h („*human readable*“) verenden. Dann bekommt man die Größen passend in kB, MB oder GB angezeigt.

```
bash > df -h
Dateisystem          Größe Benut  Verf Ben% Eingehängt auf
/dev/mapper/VolGroup00-LogVol00
                      5,2G  3,9G  1,1G  78% /
/dev/hda1              99M  9,7M   84M  11% /boot
tmpfs                 126M     0  126M   0% /dev/shm
bash >
```

Möchte man wissen, auf welchem Datenträger sich eine bestimmte Datei oder ein bestimmtes Verzeichnis befindet, kann man dazu auch das df-Kommando verwenden

```
[vaeterle@arktur ~]$ df /boot/grub/menu.lst
Dateisystem          1K-Blöcke   Benutzt Verfügbar Ben% Eingehängt auf
/dev/hda1               101086      9854     86013  11% /boot
[vaeterle@arktur ~]$
```

Die Datei `menu.lst` befindet sich also in dem Dateisystem, das sich auf der ersten IDE-Festplatte in der ersten primären Partition befindet und das im Verzeichnis `/boot` in das UNIX-Dateisystem eingebunden ist.

du

Dateien werden auf Datenträgern normalerweise nicht zeichenweise sondern in Blöcken gespeichert. Das erleichtert den Zugriff und erhöht die Zugriffsgeschwindigkeit. Allerdings verbrauchen auch kleinste Dateien von wenigen Byte Länge einen kompletten Plattenblock. Unter Linux sind die Plattenblöcke je nach Größe des Datenträgers zwischen 4 und 64 Kilobytes groß.

Um zu sehen, wie viel Plattenplatz eine Datei verbraucht, kann man das Kommando du verwenden. Ohne Optionen zeigt du die Größe in Blöcken an, wobei die Größe der Blöcke von System zu System schwanken kann. Bei älteren UNIX Systemen umfasste ein Block 512 Bytes, bei Linux-Systemen ist es häufig ein Kilobyte (1024 Bytes).

Will man ganz sicher gehen, kann man eine der Optionen -k (Kilobytes), -m (Megabytes) oder -h (human readable) verwenden.

```
bash > echo "Kleine Datei" > klein
bash > ls -l klein
-rw-rw-r-- 1 vaeterle vaeterle 13 30. Sep 17:39 klein
bash > du klein
8       klein
bash > du -k klein
8       klein
bash > du -h klein
8,0K    klein
bash >
```

Merke: Auch die kleinste Datei braucht mindestens einen Plattenblock (hier: 8 kB)!

Die Angaben von du können aufgrund von Rundungsfehlern um einen Block vom tatsächlichen Wert abweichen.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Wendet man du auf ein Verzeichnis an, so wird rekursiv die Größe aller darin enthaltenen Unterverzeichnisse[24] angegeben. Mit der Option -a („*all*") wird auch die Größe der enthaltenen Dateien angegeben. Mit der Option -s („*summarize*") wird nur die Gesamtgröße des Verzeichnisses angegeben.

```
bash > du -k verzeichnis
20      verzeichnis/sub
73572   verzeichnis
bash > du -ak verzeichnis
8       verzeichnis/klein
73536   verzeichnis/gross
12      verzeichnis/sub/mittel
20      verzeichnis/sub
73572   verzeichnis
bash > du -sk verzeichnis
73572   verzeichnis
bash >
```

6.6 Änderungen am eigenen Benutzerkonto

Die meisten Eigenschaften des eigenen Benutzerkontos kann man nicht selbst ändern, es sein denn, man hat administrative Rechte auf dem Rechner. Zwei Dinge kann allerdings jede Benutzerin und jeder Benutzer selbst ändern

- Das Passwort
- Die Login-Shell

Das Passwort wird mit dem Kommado passwd geändert

passwd: Das eigene Passwort ändern

```
bash > passwd
Changing password for user musterma
Changing password for musterma
(current) UNIX password: (altes Passwort eintippen)
New UNIX password: (neues Passwort eintippen)
Retype new UNIX password: (neues Passwort wiederholen)
passwd: all authentication tokens updated successfully
bash >
```

[24] Nach der UNIX-Philosophie sind Verzeichnisse auch nur Dateien, aber eben spezielle Dateien, die Verweise auf andere Dateien enthalten. Trotzdem brauchen Verzeichnisse natürlich Platz auf dem Datenträger.

Zunächst muss man sein aktuelles Passwort eintippen. Damit soll verhindert werden, dass ein Fremder das Passwort in einem unbeobachteten Moment ändert.

Dann wird nach dem neuen Passwort gefragt. Da das Passwort beim eintippen nicht auf dem Bildschirm erscheint, muss man es danach gleich noch einmal angeben, um Tippfehler auszuschließen. Der Dialog kann variieren, ja nachdem welche Art von Authentisierung auf dem Rechner bzw. in dem Rechnernetz verwendet wird.

chsh: Die eigene Login-Shell ändern

Die Login-Shell kann man mit dem Kommando chsh ändern

```
bash > chsh
Changing shell for musterma.
Passwort: (Passwort eingeben)
New shell [/bin/bash]: /bin/tcsh
Shell changed.
tcsh >
```

Zunächst muss man wieder das Passwort angeben (Begründung s. oben). Danach wird nach der gewünschten Shell gefragt. Hier kann man nur die Programme angeben, die in der Datei `/etc/shells` aufgelistet sind.

```
bash > cat /etc/shells
/bin/sh
/bin/bash
/sbin/nologin
/bin/tcsh
/bin/tcsh
/bin/csh
/bin/ksh
bash >
```

id: Numerische Benutzer- und Gruppenkennungen und Gruppenzugehörigkeiten eines Benutzers auflisten

Zu den Eigenschaften, die man normalerweise nicht selbst ändern kann, gehören die numerische Benutzer ID und die Benutzergruppen, zu denen man gehört. Diese Informationen kann man sich aber zumindest mit dem Kommando `id` anzeigen lassen

```
bash > id
uid=567(musterma) gid=100(user) Gruppen=200(staff-
it),400(x2000) context=user_u:system_r:unconfined_t
bash >
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Der Benutzer musterma hat die numerische Benutzer ID 567 und gehört primär der Gruppe user an, die die numerische Gruppen ID 100 hat. Außerdem gehört er noch den Gruppen staff-it und x2000 (vermutlich ein Projektteam) an. Diese Informationen werden später bei den Zugriffsrechten auf Dateien und Verzeichnisse wichtig.

Man kann id auch einen Benutzernamen als Parameter mitgeben. Dann wird die entsprechende Information über diesen Benutzer ausgegeben

```
bash > id root
uid=0(root) gid=0(root) Grppen=0(root),1(bin),2(daemon),
3(sys),4(adm),6(disk),10(wheel)
bash >
```

Man sieht in diesem Beispiel noch etwas wichtiges: Der Benutzer root hat die User ID 0 und ist damit unumschränkter Herrscher auf diesem Rechner. Das Konto kann auch anders heißen. Entscheidend ist die User ID 0.

7 Reguläre Ausdrücke

Reguläre Ausdrücke („*regular expressions*“) werden überall dort gebraucht, wo man nach einem bestimmten Muster sucht, beispielsweise nach allen Dateinamen, die mit einem A beginnen oder nach Datumsangaben in einem Text. Dieses Kapitel dient der Vorbereitung auf die Einführung in die UNIX Textfilter in Kapitel 8 . Mit den regulären Ausdrücken alleine kann man noch nicht sehr viel anfangen.

In der Microsoft-Welt gibt es zwei reguläre Ausdrücke: * und ?. Der DOS-Befehl „`dir *.exe`“ listet alle ausführbaren Dateien im aktuellen Verzeichnis auf. Der Befehl „`copy datei0?.doc a:\`“ kopiert alle die Dateien, in deren Name anstelle des Fragezeichens irgend ein Zeichen steht, auf Diskette. Kopiert würde also die Datei `datei03.doc` ebenso wie `datei0x.doc`, aber nicht `datei012.doc`.

Unter UNIX gibt es wesentlich feinere Methoden, solche Suchmuster zu formulieren.

Suche nach bestimmten Zeichen

Der einfachste reguläre Ausdruck ist ein Punkt („.“). In einem Suchmuster steht ein Punkt für irgend ein Zeichen. Wenn man gezielt nach einem Punkt sucht, muss man den Punkt maskieren, das heißt, man muss deutlich machen, dass der Punkt nicht als Sonderzeichen interpretiert werden soll, sondern eben als Punkt. Um einen Punkt oder irgend ein anderes Steuerzeichen zu maskieren schreibt man einen Backslash („\“) davor. Es bedeuten also

Ein Backslash („\“) in einem regulären Ausdruck bedeutet, dass das folgende Zeichen wörtlich zu nehmen und nicht als Sonderzeichen zu interpretieren ist.

.	Irgend ein Zeichen
\.	Ein Punkt
\\	Ein Backslash

Schließt man in einem regulären Ausdruck eine Reihe von Zeichen in eckige Klammern ein, so bedeutet dies „Suche nach einem dieser Zeichen“. Wie immer in UNIX werden kleine und große Buchstaben unterschieden. Ist das erste Zeichen in der Klammer ein „^“, so wird nach einem Zeichen gesucht, das nicht in der Klammer enthalten ist. Mögliche Varianten sind unter anderem

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

[aeiou]	Ein kleiner Vokal
[jJnN]	Einer der Buchstaben j, J, n, oder N
[A-Z0-9]	Ein Großbuchtabe oder eine Ziffer
[^xy]	Irgend ein Zeichen, das weder „x“ noch „y“ ist

Multiplikatoren

Sucht man beispielsweise nach mehreren Ziffern oder Zeichen einer bestimmten Art, kann man einen regulären Ausdruck mit einer Art Multiplikator versehen, der signalisiert, dass das Zeichen oder die Zeichenklasse vor dem Multiplikator mehrfach gesucht wird.

In einem regulären Ausdruck steht nicht * für eine beliebige Zeichenkette, wie bei DOS und Windows, sondern die Kombination „.*“

*	Null oder beliebig viele Exemplare des vorangehenden Zeichens.	
	.*	irgend eine Zeichenfolge (auch leer)
	-*	keinen oder beliebig viele Bindestriche
	[0-9]*	keine oder beliebig viele Ziffern
+	Ein oder beliebig viele Exemplare des vorangehenden Zeichens	
	a+	Ein oder beliebig viele „a“
	[^ijkl]+	Ein oder beliebig viele Zeichen, die keine i, j, k, oder l sind
?	Kein oder ein Exemplar des vorangegangenen Zeichens	
	[0-9]?	Keine oder eine Ziffer
\{n\}[25]	Genau *n* Exemplare des vorangehenden Zeichens	
	[0-9]\{2\}	Genau zwei Ziffern
\{n,\}	Mindestens *n* Exemplare des vorangehenden Zeichens	

[25] Aus historischen Gründen gibt es zwei Arten von regulären Ausdrücken: „*normale*“ und „*erweiterte*“ (*extended regular expressions*). Bei den normalen regulären Ausdrücken müssen geschweifte und runde Klammern, sowie das Pipe-Zeichen („|“) durch Backslashes maskiert werden. Bei erweiterten regulären Ausdrücken darf man dies nicht tun. Wann welche Variante zum Einsatz kommt, hängt von der Anwendung ab (vgl den Abschnitt zu grep (Seite 71).

`\{,n\}`	Höchstens *n* Exemplare des vorangehenden Zeichens
`\{n,m\}`	Mindestens n, höchstens jedoch m Exemplare des vorangehenden Zeichens

Ankerpunkte

Möchte man nach einer Zeichenkette am Anfang oder am Ende einer Zeile suchen, geht das mit

^	Anfang der Zeile	
	^[A-Z]	Suche nach Zeilen, die mit einem Großbuchstaben beginnen
$	Ende der Zeile	
	[\.]$	Suche nach Zeilen, die mit einem Punkt enden.

Dies und Das

`\(...\)`	Gruppiert reguläre Ausdrücke	
`\n`	Steht für die *n*-te runde Klammer	
`\\|`	Entweder das Suchmuster davor oder das Suchmuster dahinter	
	\([Ss]tart\\|[Ss]top\)	Start, start, Stop oder stop

Beispiele

`[0-9]\{1,2\}\.[0-9]\{1,2\}\.`	Ein Datum wie 1.5., 3.10. oder 24.12.
<[^>]+>	Ein beliebiges SGML Tag („<irgendwas>“)
^$	Eine Leerzeile (Zeilenende folgt unmittelbar auf Zeilenanfang)
0x[0-9A-Fa-f]\{4\}	Eine vierstellige Hexadezimalzahl wie „0x12a5“ oder „0xFFFF“

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

8 Textfilter unter UNIX

Üblicherweise wird unter UNIX sowohl das Betriebssystem als auch die Anwendungen durch Textdateien konfiguriert. Das hat den Vorteil, dass die Konfigurationen, zumindest prinzipiell, menschenlesbar und mit einem einfachen Texteditor veränderbar sind. Der Nachteil bei diesem Verfahren ist ein relativ großer Platzbedarf der Konfigurationsdateien und eine bei großen Dateien mäßige Leistungsfähigkeit. Bei normale Arbeitsplatzrechnern überwiegen allerdings die Vorteile.

Deshalb bietet UNIX eine ganze Reihe von Dienstprogrammen, die zur Manipulation von Textdateien geeignet sind. Die Programme zur Ausgabe von Dateien (`cat`, `more`/`less`, head, `tail` und od) wurden bereits in Abschnitt 6.3 ab Seite 35 erläutert.

8.1 Textdateien sortieren

Das Kommando sort sortiert die Zeilen einer oder mehrerer Dateien alphabetisch oder numerisch. Wird keine Datei oder ein Bindestrich als Dateiname angegeben, so werden die zu sortierenden Zeilen vom stdin-Kanal gelesen.

```
bash > sort /etc/passwd
```

sortiert die Zeilen der Passwort-Datei alphabetisch.

Das sort-Kommando kennt folgende wichtige Optionen

-n Sortiert nicht alphabetisch sondern numerisch

```
bash > du -ak /sbin | sort
```

sortiert die Dateien in `/sbin` der Größe nach – aber falsch (alphabetisch). Das heißt, eine 9 kB große Datei wird nach einer Datei von 17 kB einsortiert.

```
bash > du -ak /sbin | sort -n
```

sortiert die Zahlen am Anfang der einzelnen Zeilen richtig, das heißt, der Größe nach.

`-r` Kehrt die Reihenfolge der Sortierung um.

```
bash > du -ak /sbin | sort -rn
```

Sortiert wiederum die Dateien in `/sbin` der Größe nach. Jetzt kommen aber die größten Dateien zuerst.

`-k` Ermöglicht die Sortierung nach einer anderen Spalte als der ersten.

Möchte man beispielsweise kleine Dateien, die laut „du“ alle einen einzigen Plattenblock belegen, der Größe nach sortieren, kann man das `ls`-Kommando verwenden. Bei einem langen Listing steht die Dateigröße allerdings in der fünften Spalte

```
bash > ls -l /sbin | sort -r -n -k 5,5
```

Die Sortierung erstreckt sich „*von Spalte 5 bis Spalte 5*“

`-t` Erlaubt die Definition eines Spaltentrenners

Die einzelnen Spalten der Passwort-Datei werden durch Doppelpunkte getrennt. Möchte man die Einträge nach aufsteigender numerischer Benutzer-ID (dritte Spalte) sortieren, kann man dies mit der Befehlszeile

```
bash > sort -t ':' -k 3 -n /etc/passwd
```

`-u` („*unique*“) wenn sich bei den sortierten Zeilen eine Zeile mehrfach wiederholt, wird dies nur einmal ausgegeben

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

8.2 Suchen nach Strings

Das Kommando grep sucht in einer oder mehreren Dateien nach einer Zeichenkette. Dieses Kommando kann in drei Varianten verwendet werden

grep -F	fgrep	Sucht nur nach konstanten Strings
grep		Sucht nach konstanten Zeichenketten und regulären Ausdrücken
grep -E	egrep	Sucht nach konstanten Zeichenketten und erweiterten regulären Ausdrücken

```
bash > fgrep udp /etc/services
```

gibt alle Zeilen der Datei /etc/services aus, die die Zeichenkette udp enthalten.

```
bash > last | grep '\(Aug\|Sep\)'
```

Zeigt an, wer sich im August oder September am lokalen System angemeldet hat. Da es sich hier um „gewöhnliche“ reguläre Ausdrücke handelt, müssen die Klammern und das Pipe-Zeichen maskiert werden. Verwendet man „erweiterte“ reguläre Ausdrücke, kann (und muss) man auf die Maskierung verzichten.

```
bash > last | grep -E '(Aug|Sep)'
```

Alle Spielarten von grep kennen unter anderem folgende Optionen

-i Unterscheidet bei der Suche nicht nach Klein- und Großbuchstaben

-c Gibt nur an, wie oft die Suche erfolgreich war

-l Gibt nur die Namen der Dateien aus, in denen die gesuchte Zeichenkette gefunden wurde

-v Kehrt die Suche um, d.h. gibt die Zeilen aus, in denen die gesuchte Zeichenkette **nicht** enthalten ist

-n Gibt zu jeder Fundstelle die Zeilennummer aus

8.3 Texte filtern

cut

Das grep-Kommando filtert Texte zeilenweise. Das cut-Kommando kann aus Dateien einzelne Spalten ausschneiden. Dabei können die Spalten verschieden definiert sein.

Im einfachsten Fall durch die Position eines Zeichens in der Zeile

```
bash > echo ”123456789:123456789 123456789“ > cutter
bash > cut -b 5-15 cutter
56789:12345
bash >
```

Alternativ zur Option -b („*Byte*“) kann auch die Option -c („*Character*“) verwendet werden.

Alternativ kann man auch „Felder“ (Option -f) aus den Zeilen ausschneiden, die durch bestimmte Trennzeichen (Option -d („*Delimiter*“)) getrennt werden

```
bash > cut -d ':' -f 2 cutter
123456789 123456789
bash >
```

Übung:

Schneiden Sie aus der Passwort-Datei das erste (Username) und fünfte Spalte (wirklicher Name des Benutzers) aus. Der Feldtrenner ist ein Doppelpunkt.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

tr

Das Kommando `tr` („*transpose*“) ersetzt jedes Zeichen aus einer Zeichenkette durch das Zeichen, das in einer zweiten Zeichenkette an derselben Position steht. Das tr-Kommando liest nur von stdin und schreibt nur nach stdout

```
bash > echo ”Nur ein kleiner Test” | tr ’en’ ’XY’
Nur XiY klXiYXr TXst
bash >
```

Das tr-Kommando kann auch verwendet werden, um bestimmte Zeichen aus einem Zeichenstrom zu löschen

```
bash > echo ”Nur ein kleiner Test” | tr -d ’en’
?Nur i klir Tst?
bash >
```

Übung:

Das `tr` Kommando lässt sich dazu verwenden, Texte mit einer ganz bestimmten Art von Verfahren, den so genannten Caesar-Codes, zu verschlüsseln. Bei diesen Codes wird jeder Buchstabe einer Nachricht durch einen anderen Buchstaben ersetzt, der eine bestimmte Anzahl von Positionen weiter hinten im Alphabet steht. Das „Geheimnis“ des Codes besteht darin, um wie viele Positionen der Code gegenüber dem originalen Zeichen verschoben ist. Der Caesar-Code ist zwar ziemlich leicht zu knacken, ist aber gut genug,, den Inhalt einer Botschaft für den flüchtigen Betrachter zu verschleiern.

Bei einer Verschiebung von 13 Zeichen (dem halben Alphabet) hat der Code eine Besonderheit: Verschlüsselung und Entschlüsselung funktionieren identisch.

```
Klartext:    ABCDEFGHIJKLMNOPQRSTUVWXYZ
             ||||||||||||||||||||||||||
Code:        NOPQRSTUVWXYZABCDEFGHIJKLM
```

Aus dem Wort „GEHEIM“ würde danach „TRURVZ“. Diese Verschlüsselung, die auch als ROT13 Verschlüsselung („Rotation des Alphabets um 13 Zeichen“) bezeichnet wird, war in den Anfangszeiten des Internets in so genannten Usenet Newsgroups beliebt, um Inhalte zu

verschleiern, die manche zufällige Leser als anstößig empfinden könnten. Eine einfache ROT13-Verschlüsselung unter UNIX könnte so aussehen:

```
bash > echo „Ein supergeheimer Text“ |\
tr ”ABC...XYZabc...xyz” ”NOP...KLMnop...klm”
Rva fhcretrurvzre Grkg
bash > echo ”Rva fhcretrurvzre Grkg” |\
tr ”ABC...XYZabc...xyz” ”NOP...KLMnop...klm”
Ein supergeheimer Text
bash >
```

Der Backslash am Ende der ersten Kommandozeile maskiert das Zeilenende, so dass die folgende Zeile auch noch als Teil des ersten Kommandos interpretiert wird.

sed

Der „*Stream Editor*“ sed ist ein Werkzeug um komplexe Filteroperationen an Textdateien vorzunehmen. Die Anweisungen, was sed mit dem eingegebenen Text tun soll, können entweder in einer Skript-Datei gespeichert werden oder direkt auf der Kommandozeile übergeben werden.

```
bash > sed -f bla.sed datei1 datei2
```

Die Dateien datei1 und datei2 werden von sed zeilenweise eingelesen. Auf jede Zeile werden nacheinander alle Kommandos, die in der Skriptdatei bla.sed stehen, angewendet. Das Resultat wird auf der Standard-Ausgabe ausgegeben.

Möchte man nur ein einziges Kommando oder einige wenige Kommandos auf die eingelesenen Dateien anwenden, so können die auch direkt auf der Kommandozeile übergeben werden. Ein Beispiel:

```
bash > sed -e 's/FHTE/Hochschule Esslingen/' datei1 datei2
```

In diesem Beispiel werden die Dateien datei1 und datei2 eingelesen und jedes Vorkommen der Zeichenkette „*FHTE*“ wird durch die Zeichenkette „*Hochschule Esslingen*“ ersetzt.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Die besondere Stärke von sed liegt darin, dass man in den sed-Kommandos auch reguläre Ausdrücke verwenden kann.

Die Kommandos für sed bestehen aus einer optionalen Adresse und der eigentlichen Anweisung. Die Adresse kann entweder eine einfache Zeilennumnmer oder ein regulärer Ausdruck als Suchmuster sein. Das Zeichen „$“ als Adresse bedeutet, dass die Anweisung nur auf die letzte Zeile angewandt werden soll. Wird keine Adresse angegeben, wird das betreffende Kommando auf jede Eingabezeile angewandt.

sed-Kommando s (substitute)

```
[Adresse]s/<Suchmuster>/<Ersatz>/[flags]
```

Dieses Kommando ersetzt in den Zeilen, auf die die Adresse passt, die Zeichenkette, auf die das Suchmuster passt, durch die Zeichenkette <Ersatz>.

Das wichtigste Flag ist „g“ (global). Normalerweise wird nur das erste Vorkommen des Suchstrings in einer Zeile ersetzt. Mit diesem Flag werden alle Treffer in einer Zeile ersetzt.

Üblicherweise werden das Kommando, das Suchmuster und die Ersatzzeichenkette durch Schrägstriche „/“ getrennt. Taucht im Suchstring oder in der Ersatzzeichenkette ein „/“ auf, so muss dieses Zeichen durch einen Backslash maskiert werden. Im Prinzip kann statt des Schrägstrichs auch jedes andere Zeichen verwendet werden. sed interpretiert das erste Zeichen, das auf das Kommando folgt, als Trennzeichen. Statt

```
s/\/usr\/local/\/opt/g
```

kann man also übersichtlicher

```
s!/usr/local!/opt!g
```

schreiben.

Als Vorbereitung für die nächste Übung sollten Sie eine Textdatei erzeugen, die Telefonnummern enthält. Also zum Beispiel

```
telefon.txt:
(0711) 397-4211
(0123) 456789    (0800) 112233
(030) 98765
```

Diese Telefonnumern sollen nun in das international gebräuchliche Format „+49 (0) ...“ umformatiert werden. Das entsprechende Kommando wäre

```
bash > sed -e 's/(0\([0-9]*\))/\+49-(0) \1/' telefon.txt
+49 (0) 711 397-4211
+49 (0) 123 456789    (0800) 112233
+49 (0) 30 98765
```

Wie bei der Einführung der regulären Ausdrücke (ab Seite 52) schon erwähnt, kann man Teile des Suchstrings durch maskierte runde Klammern „\(... \)“ gruppieren und später in der Ersatzzeichenkette durch „\1“, „\2“ etc. auf das erste, zweite und die weiteren Gruppen zugreifen. In diesem Fall wird die Vorwahl ohne führende Null gespeichert, um sie später hinter der Klammer ausgeben zu können.

Bei genauer Betrachtung stellt man fest, dass in der zweiten Zeile nur eine Nummer geändert wurde. Wenn alle Nummern geändert werden sollen, müsste das sed-Kommando noch um die Option „g“ erweitert werden.

sed-Kommando y (replace)

[Adresse]y/<String1>/<String2>/

Dieses Kommando ersetzt in den Eingabedateien jedes Zeichen in <String1> durch das an der entsprechenden Stelle der zweiten Zeichnkette <String2> stehende Zeichen. Damit verhält sich sed wie das auf Seite 59 diskutierte Kommando tr.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Mit dem Kommando

```
bash > sed -e 'y/ABCDEFGHIJKLMNOPQRSTUVWXYZ\
/NOPQRSTUVWXYZABCDEFGHIJKLM/ klartext.txt
```

kann man genau wie mit dem tr-Kommando eine ROT13-Verschlüsselung realisieren.

Weitere sed-Kommandos

`/<Suchmuster>/d`	Löscht alle Zeilen, in denen das Suchmuster gefunden wird
`[Adresse]a\ Text ...`	Hinter der durch [Adresse] identifizierten Zeile wird „Text ...“ eingefügt
`[Adresse]i\ Text ...`	Vor der durch [Adresse] identifizierten Zeile wird „Text ...“ eingefügt
`[Adresse]c\ Text ...`	Die durch [Adresse] identifizierte Zeile wird durch „Text ...“ ersetzt
`[Adresse]p`	Gibt die aktuelle Zeile der Eingabedateien aus (Nützlich zum Beispiel, wenn man Zeilen kopieren möchte).
`[Adresse]n`	Liest die nächste Zeile ein, auf die die Adresse passt, ohne das Skript neu zu starten
`[Adresse]r <Datei>`	Liest den Inhalt der Datei <Datei> ein und gibt ihn auf stdout aus.
`[Kommando]w <Datei>`	Schreibt das Ergebnis des sed-Kommandos [Kommando] in die datei <Datei>

Übungsaufgabe

Mit sed soll ein Filter realisiert werden, der einen Text wie den folgenden in ein HTML-Dokument verwandelt:

```
1 Textbearbeitung mit UNIX

Einführender Text, der beliebig lang sein kann

2 Reguläre Ausdrücke
2.1 Gewöhnliche reguläre Ausdrücke

Alles über reguläre Ausdrücke, Aufzucht und Pflege, ...

2.2 Erweiterte reguläre Ausdrücke

Alles über die erweiterten regulären Ausdrücke
```

Es müssen im Einzelnen folgende Aufgaben gelöst werden:

- Ausgabe eines HTML-Kopfes vor dem eigentlichen Text
- Formatierung der Überschriften: Beginnt die Zeile mit einer Ziffer, so handelt es sich um eine Kapitelüberschrift („<h1>“), beginnt die Zeile mit einer Ziffer, einem Punkt und wieder einer Ziffer, so handelt es sich um eine Unterkapitelüberschrift („<h2>“).
- Nach der letzten Textzeile muss ein HTML-Fuß eingefügt werden.

Diese Aufgaben erfüllt das folgende sed-Skript

```
1i\
<html>\
<head>\
</head>\
<body>
/^[0-9] /s/\(^.*\)/<h1>\1<\/h1>/
/^[0-9]\.[0-9] /s/\(^.*\)/<h2>\1<\/h2>/
$a\
</body>\
</html>
```

Schreibt man diese sed-Anweisungen in eine Datei und nennt diese

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

z.B. „markup.sed“, so kann man den oben angegebenen Text mit dem Kommando

```
bash > sed -f markup.sed text.txt
```

in eine HTML-Datei umwandeln.

awk

Das Programm awk ist ein Interpreter für die gleichnamige Programmiersprache, die auf die Formatierung und Filterung von Textdateien spezialisiert ist. In mancher Hinsicht ähnelt awk dem eben besprochenen sed, ist aber wesentlich mächtiger. Wie es sich für eine echte Programmiersprache gehört, kennt awk Variablen, Schleifen, bedingte Verzweigungen und logische Operatoren.

Wie bei sed kann man auch awk-Anwendungen entweder in einer Skriptdatei ablegen

```
bash > awk [Optionen] -f skript.awk <Datei1> <Datei2> ...
```

oder die Anweisungen direkt auf der Kommandozeile übergeben

```
bash > awk [Optionen] '<awk-Anweisungen>' <Datei1> <Datei2>
...
```

Die Ausgabe der Resultate erfolgt auf stdout, kann aber natürlich in eine Datei umgelenkt werden. Die wichtigsten Optionen, die man awk mitgeben kann, sind

`-v VNAME=Wert`	Setzt die Variable VNAME auf „Wert“.
`-F '&'`	Setzt den „Field Separator“, der die einzelnen „Worte“ in einer Zeile trennt, auf „&“ (der Standard ist das Leerzeichen und/oder der Tabulator)

Jedes awk-Skript hat prinzipiell drei Teile. Der erste Teil wird nur einmal zu Beginn der Abarbeitung des Skriptes ausgeführt. Die Anweisungen des zweiten Teils werden nacheinander auf alle Zeilen der

Eingabedateien angewandt. Der dritte Teil des Skriptes schließlich wird nur einmal am Ende der Ausführung des Skriptes ausgeführt. Abbildung 8.1 veranschaulicht dies.

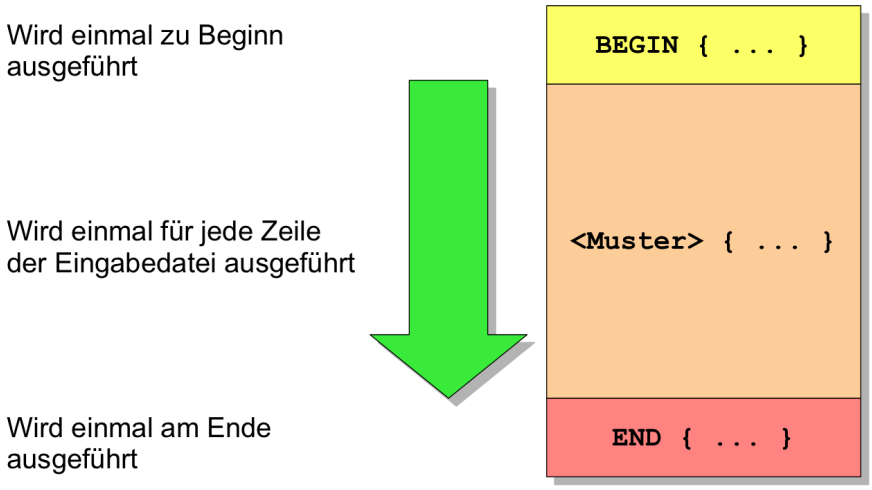


Abbildung 8.1: Schematischer Ablauf eines awk-Skriptes

Ein Beispiel dazu:

```
BEGIN { FS=":"
printf ( "+------------+-------+-------+\n" )
printf ( "|    User    |  UID  |  GID  |\n" )
printf ( "+------------+-------+-------+\n" )
}
{ printf ( "| %10s | %5s | %5s |\n", $1, $3, $4 ) }
END { printf ( "+------------+-------+-------+\n" )}
```

Dieses awk-Skript, angewandt auf die UNIX Passwort-Datei /etc/passwd, gibt Benutzername, UID und primäre GID der Benutzer des aktuellen Systems an.

Im ersten Teil des Skriptes, der durch das Schlüsselwort BEGIN eingeleitet wird, wird zunächst der „Field Separator“ auf den Doppelpunkt geändert. In den folgenden Zeilen wird eine Ausgabe auf der Konsole veranlasst. Dazu wird das von C her bekannte Kommando printf verwendet. Im Unterschied zu C müssen die Zeilen allerdings nicht durch Strichpunkte abgeschlossen werden.

Der zweite Teil des Skriptes (das ist der Inhalt der zweiten geschweiften Klammer) wird für jede Zeile der Eingabedatei, in diesem Fall also /etc/passwd, ausgeführt. Das bedeutet, für jede Zeile der Passwort-

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

datei wird das printf-Kommando ausgeführt, das das erste, dritte und vierte Wort der Zeile, also den Benutzernamen, die UID und die primäre GID formatiert ausgibt. Dieser Teil des Skriptes kann sich beliebig oft wiederholen

Der letzte Teil des Skriptes, eingeleitet durch das Schlüsselwort END, gibt noch den Rest des Rahmens aus.

Um das Skript bei der Arbeit zu sehen, muss man das Kommando

```
bash > awk -f passwd.awk /etc/passwd
```

absetzen. In der Datei passwd.awk sind die oben genannten awk-Anweisungen enthalten.

Variable können an jeder Stelle des awk-Skriptes durch einfache Zuweisung („VNAME=Wert“) definiert werden. Der Zugriff auf eine Variable erfolgt einfach über ihren Namen. Einige Variable sind standardmäßig gesetzt und können einem das Leben wesentlich vereinfachen

FS	(„Field Separator“) Feldtrenner zwischen den „Worten“ in einer Zeile. Standard: Leerzeichen und/oder Tabulator
RS	(„Record Separator“) Trenner zwischen den Zeilen der Eingabedateien. Standard: Newline
NR	(„Number of Record“) Nummer der aktuellen Zeile
NF	(„Number of Fields“) Zahl der „Worte“ in der aktuellen Zeile
$n	n-tes Feld der aktuellen Zeile
$0	Die aktuelle Zeile
OFS	(„Output Field Separator“)
ORS	(„Output Record Separator“)
FILENAME	Name der Datei, aus der die aktuelle Eingabezeile

stammt

Zu jeder Programmiersprache gehören Schleifen und bedingte Verzweigungen. awk kennt drei Arten von Schleifen

- while (<Bedingung>) <Kommando>
- do <Kommandos> while (<Bedingung>)
- for (<set>; <test>; <increment>) <Kommando>

Sollen mehrere Kommandos innerhalb der Schleife ausgeführt werden, müssen die durch geschweifte Klammern zu einer Gruppe zusammengefasst werden.

Für bedingte Verzweigungen kennt awk zwei Varianten

- if (<Bedingung>) <Kommando>
  [else <Anderes Kommando>]
- <Ausdruck> ? <entweder> : <oder>

Details zu den einzelnen Konstrukten finden sich in der Manpage von awk oder in der einschlägigen Fachliteratur[26].

Das abschließende Beispiel zeigt, dass awk eine vollwertige Programmiersprache ist, die zwar prinzipiell auf die Formatierung und Filterung von Texten spezialisiert ist, prinzipiell aber auch für andere Zwecke eingesetzt werden kann. Das folgende Skript berechnet die Fakultät zu einer eingegebenen Zahl. Wird eine Eingabe gemacht, die keine (Dezimal-) Zahl ist, wird eine Fehlermeldung ausgegeben:

```
#!/bin/awk -f
#
# --- fakultaet.awk:
#
BEGIN { printf( "Zahl eingeben :" ) }
$1 ~ /^[0-9]+$/ {
number = $1
if ( number == 0 )
```

[26] Besonders empfohlen sei für Neugierige das Buch „sed and awk“ von Dale Dougherty und Arnold Robbins, erschienen bei O’Reilly & Associates. Wer es kürzer mag: „sed und awk. Kurz und gut“ von Arnold Robbins (ebenfalls bei O’Reilly erschienen).

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

```
fact = 1
else
fact = number
for ( x = number - 1; x > 1; x-- )
fact *= x
printf ( "Die Fakultaet von %d ist %g\n", number, fact )
exit
}
{ printf ( "Ungueltige Eingabe, bitte ZAHL eingeben :" ) }
```

Wie bei Shellskripten leitet das „#“ einen Kommentar ein, das heißt, der Interpreter ignoriert alles, was hinter einem „#“ steht bis zum nächsten Zeilenende.

Die erste Zeile des Skriptes enthält eine „magische“ Zeichenfolge („#!“). Macht man die Datei ausführbar („chmod +x ...“), interpretiert die aufrufende Shell alles, was hinter diesen beiden Zeichen steht als Pfad zu einem Programm, das in der Lage ist, das folgende Skript zu interpretieren. Auf diese Weise kann man das Skript einfach durch Aufruf seines Namens starten und spart sich das „awk –f ...“. Es heißt dann

```
bash > ./fakultaet.awk
```

statt

```
bash >  awk -f ./fakultaet.awk
```

was aber trotzdem immer noch möglich wäre.

Die Antworten finden Sie auf Seite 97. Aber versuchen Sie in jedem Fall erst einmal, selbst auf die Lösung zu kommen und schauen Sie im Zweifelsfall noch einmal im Text nach. Durch „Schummeln“ schaden Sie niemanden – außer sich selbst.

Fragen zur Sebstkontrolle

1. Mit welcher UNIX Kommandozeile können Sie feststellen, auf welcher Festplatte das Verzeichnis `/home/musterma` liegt?

2. Mit welcher UNIX Kommandozeile könne sie feststellen, wie viel Platz das Verzeichnis `/data/mp3` auf der Festplatte in Anspruch nimmt?

3. Mit welchem erweiterten regulären Ausdruck können Sie in einer Datei Daten im ISO 8601-Format (Vier Ziffern, Bindestrich, zwei Ziffern, Bindestrich, zwei Ziffern, Beispiel: 2006-12-31) finden?

4. Warum funktioniert dieser reguläre Ausdruck nicht in der Kommandozeile „`grep '<regexp>' datei`“, wobei `<regexp>` für die Antwort auf Frage 3 steht?

5. Was müssen Sie tun, damit die Kommandozeile funktioniert?

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

6. Geben Sie die UNIX Kommandozeile an, mit der Sie in einer Textdatei mehrere aufeinander folgende Leerzeilen durch eine einzige und mehrere aufeinander folgende Leerzeichen durch ein einziges ersetzen können. Um diese Frage zu beantworten, müssen Sie eventuell die man-Pages zu den Befehlen `cat` und `tr` studieren!

7. Mit welchem sed-Kommando können Sie aus einem Text alle Zeilen löschen, die mit einem „#“ beginnen?

8. Mit welchem sed-Kommando können Sie in einem Text alle Daten in dem in Deutschland gebräuchlichen Format („24.12.2006“) in das ISO 8601-Format („2006-12-24“) verwandeln?

9 Bash für Profis

9.1 Spielarten von Shells

Die Hauptaufgabe einer Shell besteht, wie gesagt, darin, Eingaben der Benutzerin bzw. des Benutzers zu interpretieren und Ausgaben der Programme auf dem Terminal darzustellen. Solche Shells nennt man *interaktive* Shells. Interaktive Shells müssen mit einem realen oder virtuellen Terminal, also einer entsprechenden Hardware oder einem Fenster einer graphischen Benutzerschnittstelle, verbunden sein.

Vor allem bei der Ausführung von Shellskripten kann es aber auch vorkommen, dass eine Shell nicht mit einem Terminal verbunden ist, also keine Eingaben entgegennehmen und keine Ausgaben machen kann. Solche Shells heißen *nicht-interaktive* Shells.

Ob eine Shell interaktiv ist, oder nicht, kann man auf einem Linux-System auf zwei Arten herausfinden:

1. Nach Eingabe des Kommandos `tty` liefert eine interaktive Shell den Namen der Spezialdatei[27], die das verbundene Terminal repräsentiert. Bei nicht-interaktiven Shells wird nichts zurückgegeben.

2. Die Systemvariable $-, in der die Optionen gespeichert sind, mit denen die aktuelle Shell gestartet wurde, enthält bei interaktiven Shells ein kleines i. Bei einer interaktiven Shell würde man beispielsweise folgendes beobachten

```
bash > echo $-[28]
himBH
```

[27] Wie in der Vorlesung noch dargelegt werden wird, werden auf UNIX Systemen die Hardwarekomponenten durch spezielle Dateien dargestellt. Dadurch reduziert sich die Ansteuerung von Hardware auf das Lesen aus und das Schreiben in Dateien.

[28] Außer der gesuchten Option –i sind standardmäßig noch weitere Optionen gesetzt: -h (Merkt sich, unter welchem Pfad einmal aufgerufene Programme liegen), -m (Überwacht Hintergrundprozesse und gibt beispielsweise eine Meldung aus, wenn einer davon beendet wurde, -B (Aktiviert die so genannte „Brace Expansion“ (s. Manpage)) und –H (statt des Kommandos history kann auch ein Ausrufezeichen eingegeben werden)

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

```
bash >
```

Streng genommen weiß man natürlich in dem Moment, in dem man der Shell per Tastatur etwas übergeben kann, dass es sich um eine interaktive Shell handelt. Die genannten Methoden sind auch eher für den Einsatz in Shellskripten gedacht, wenn man beispielsweise überprüfen will, ob eine Ausgabe auf dem Bildschirm möglich ist oder nicht.

Man kann die bash gezielt als nicht-interaktive Shell aufrufen, indem man ihr die Option –c mitgibt. In diesem Fall muss man das Kommando, das die Shell ausführen soll, natürlich mitgeben. Also zum Beispiel

```
bash > bash -c 'echo $-'
hBc
bash >
```

Die aufgerufene bash führt das bereits im vorigen Beispiel verwendete Kommando „echo $-" aus, um die Optionen der aktuellen Shell auszugeben. Die einfachen Anführungszeichen sorgen dafür, dass das echo Kommando und seine Argumente auch wirklich von der aufgerufenen Shell interpretiert werden und nicht schon von der aufrufenden. Die Option –c macht die Shell zur nicht-interaktiven Shell. Die Ausgabe der bash enthält nur noch die Buchstaben „hBc" (s. Fußnote auf der vorigen Seite). Das heißt, die Shell, die das echo-Kommando ausgeführt hat, war tatsächlich nicht-interaktiv.

Bei der bash gibt es noch eine dritte Variante der Shell, die es so bei der originalen Bourne-Shell nicht gab: die Login-Shell. Eine solche Login-Shell wird gestartet, wenn man sich auf einem Rechner neu anmeldet. Durch diese Unterscheidung ist es beispielsweise möglich, bei der erstmaligen Anmeldung automatisch zu überprüfen, ob neue Mails angekommen sind, während diese Überprüfung beim Öffnen eines neuen Konsolenfensters (interaktive Shell) nicht notwendig ist.

Mit der Option `-l` oder `--login` kann man gezielt eine Login-Shell starten. Man erkennt eine Login-Shell daran, dass die Ausgabe des Kommandos „echo $0“ mit einem Minuszeichen beginnt[29]

9.2 Startup-Dateien

Wird eine bash gestartet, so werden, je nachdem, ob es sich um eine nicht-interaktive, eine interaktive oder eine Login-Shell handelt, verschiedene Shellskripte abgearbeitet. In diesen Dateien kann man Abkürzungen oder Aliasnamen für Kommandos definieren, man kann das Verhalten der Shell beeinflussen oder man kann bestimmte Kommandos ausführen, um beispielsweise nach neuer E-Mail zu schauen.

Diese Dateien sind teils systemweit gültig, teils aber auch benutzerspezifisch. Die systemweit gültigen Dateien liegen im Verzeichis `/etc`, die benutzerspezifischen Dateien im Heimatverzeichnis des Benutzers. Die Namen der benutzerspezifischen Dateien beginnen mit einem Punkt. Das führt dazu, dass diese Dateien normalerweise nicht angezeigt werden, wenn man den Inhalt des Verzeichnisses anzeigen lässt, in dem die Datei liegt. In der Microsoft-Welt würde man dieses Verhalten durch das Setzen des „Versteckt“-Attributs der Datei erreichen.

.bashrc (Benutzer)

In dieser Datei werden alle Aktionen zusammengefasst, die beim Start einer Shell ablaufen sollen, die keine Login-Shell ist. Da diese Datei auch bei der Ausführung von Shellskripten ausgeführt wird, sollte man auf Bildschirmausgaben in dieser Datei verzichten. Solche Kommandos gehören in die profile-Dateien.

---

[29] Die Umgebungsvariable $0 enthält den Namen, unter dem die aktuelle Shell aufgerufen wurde. Bei der aktuellen Version der bash funktioniert das allerdings nicht immer. Startet man die bash mit der Option –l bzw. --login, so liefert echo $0 „bash“. Die Shell ist trotzdem eine Login-Shell, was man leicht nachweisen kann, indem man eine Bildschirmausgabe in $HOME/.profile einfügt. Startet man aber beispielsweise ein X-Windows Terminal mit der Option –ls (Login Shell), tippt also „xterm –ls“ ein, so liefert das Kommando (im neuen Terminalfenster eingegeben) „echo $0“ wie erwartet „-bash“. Offenbar ein Bug in der bash?

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

`/etc/profile` (System), `.bash_profile`, `.bash_login` oder `.profile` (Benutzer)

Diese Dateien werden nur beim Start einer Login-Shell ausgeführt. In diesen Dateien werden Aktionen definiert, die bei der ersten Anmeldung an einen Rechner ausgeführt werden sollen. Sinnvollerweise enthalten diese Dateien alle Aktionen, die eine Bildschirmausgabe erfordern. Dass die benutzerspezifische Datei drei Namen haben kann, hat historische Gründe[30]. Die bash sucht in der angegebenen Reihenfolge nach diesen Dateien und führt nur die erste existierende und lesbare Datei aus.

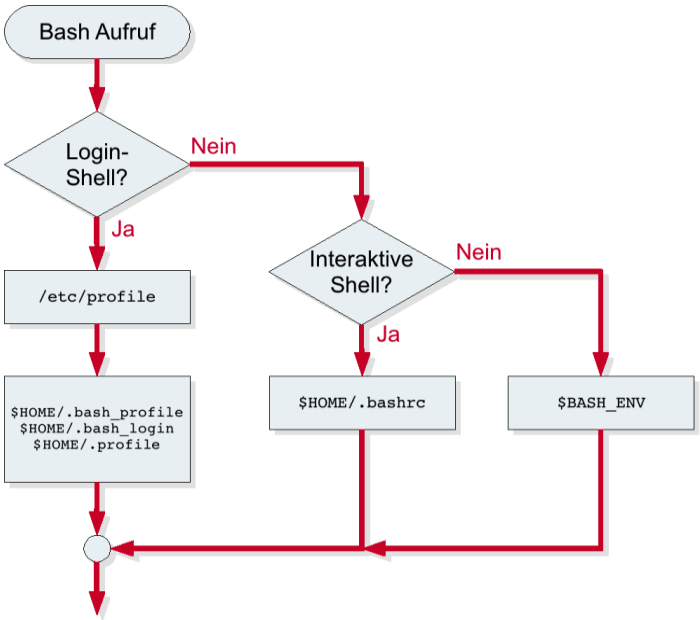


Abbildung 9.1: *Schematische Darstellung der Abarbeitung der verschiedenen Startup-Dateien der bash.*

Eine **Login-Shell** liest die systemweite Datei `/etc/profile` und die erste verfügbare der drei benutzerspezifischen Dateien `.bash_profile`, `.bash_login` und `.profile` ein. Eine **interaktive Shell**, die nicht Login-Shell ist, liest die benutzerspezifische Datei `.bashrc` ein. Eine **nicht-interaktive Shell** schließlich wertet die Umgebungsvariable $BASH_ENV aus und führt, wenn möglich, die dort genannte Datei aus.

---

[30] Schon vor der bash gab es UNIX Shells, die interaktive und Login-Shells unterscheiden konnten. Die entsprechenden Startdateien hatten (natürlich) unterschiedliche Namen: .profile (ksh) oder .login (csh bzw. tcsh).

Auffällig ist, dass es zu der benutzerspezifischen Datei `./bashrc` keine systemweit gültige Entsprechung gibt. Bei den gängigen Distributionen findet man die Datei `/etc/bashrc` aber vor. Des Rätsels Lösung: Diese Datei wird bei der Ausführung der anderen Startup-Dateien „gesourced“, das heißt, die in `/etc/bashrc` enthaltenen Kommandos werden so ausgeführt, als ob sie in der aufrufenden Datei enthalten gewesen wären.

Die Antworten finden Sie auf Seite 98. Aber versuchen Sie in jedem Fall erst einmal, selbst auf die Lösung zu kommen und schauen Sie im Zweifelsfall noch einmal im Text nach. Durch „Schummeln“ schaden Sie niemanden – außer sich selbst.

Fragen zur Sebstkontrolle

1. Welche drei Varianten der bash Shell gibt es?

2. Woran erkennt man eine interaktive Shell?

3. Wie kann man gezielt eine Login-Shell starten?

4. Welche Startup-Datei(en) wird/werden ausgeführt, wenn man eine interaktive bash startet?

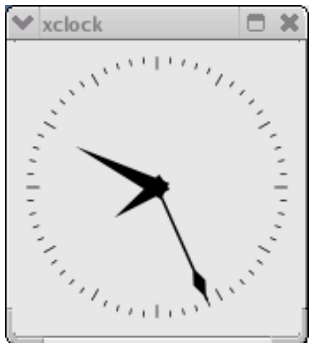
© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

9.3 Jobkontrolle

Führt man in der Shell ein Kommando aus, so ist die Shell normalerweise für weitere Eingaben blockiert, bis das Kommando abgeschlossen ist. Wenn das Programm lange läuft, wie beispielsweise ein Backup-Programm, ist das natürlich ärgerlich, weil man in dieser Zeit nichts anderes tun kann. Deshalb kann man Kommandos in der Shell auch „im Hintergrund“ laufen lassen. Man spricht auch von *Hintergrundprozessen.*

Das nächste Experiment setzt voraus, dass man auf einer X-Windows-Oberfläche arbeitet und die Shell in einem Terminalfenster läuft. Als erstes wird eine virtuelle Uhr gestartet[31]

```
bash > xclock -upd 1
```



Die Option ”-upd 1“ sorgt dafür, dass die Uhr einen Sekundenzeiger hat und jede Sekunde aktualisiert wird.

Während der Sekundenzeiger nun fröhlich seine Runden zieht, bleibt die Shell im Terminalfenster unbenutzbar.

Man kann die Shell allerdings leicht befreien, in dem man gleichzeitig die [Strg]- bzw. [Ctrl]-Taste[32] und den Buchstaben z drückt. Nun sieht das Terminalfenster so aus

```
bash > xclock -upd 1

[1]+  Stopped        xclock -upd 1
bash >
```

Die Tastenkombination [Ctrl]-z bzw. [Strg]-z hält das aktuell laufende Programm an und gibt das Terminal an die aufrufende Shell zurück.

Was ist passiert? Das Zeichen „Ctrl-z“ war das Signal an das Betriebs-

---

[31] Wenn das Programm xclock nicht installiert ist, können Sie es nachinstallieren. Auf Fedora-Distributionen verwenden Sie dazu entweder die graphischen Programme zur Softwareverwaltung oder auf einer Kommandozeile das Kommando „`yum install xclock`“.

[32] Auf englischen Tastaturen heißen die beiden Tasten in den unteren Ecken „Ctrl“ (kurz für „Control“). Auf deutschen Tastaturen heißen die Tasten „Strg“ (kurz für „Steuerung“). Im weiteren wird nun das üblichere englische „Ctrl“ verwendet.

system, die Ausführung des xclock-Kommandos anzuhalten und die Kontrolle über das Terminalfenster wieder an die aufrufende Shell zu übergeben. Der Sekundenzeiger der Uhr steht jetzt. Man kann jetzt zwar wieder mit dem Terminal arbeiten, weiß aber nicht, wie spät es ist.

Besserung bringt ein eingebautes Kommando der Shell: bg („*background*“):

Das eingebaute Shellkommando „bg“ startet einen angehaltenen Prozess im Hintergrund

```
bash > xclock -upd 1 ... Crtl-z drücken ...

[1]+  Stopped         xclock -upd 1
bash > bg
[1]+ xclock -upd 1 &
bash >
```

Die Uhr läuft wieder und die Eingabeaufforderung der Shell ist trotzdem da. Aus Sicht der Shell ist die xclock nun wie ein guter Butler: unsichtbar im Hintergrund, aber immer im Dienst.

Ein mit „&“ abgeschlossenes Kommando wird sofort im Hintergrund gestartet

Man könnte nun noch eine zweite Uhr mit rotem Hintergrund starten. Wenn man von vorneherein weiß, dass man eine Anwendung im Hintergrund ausführen möchte, kann man das der Shell mitteilen, indem man die Kommandozeile mit einem *Ampersand* („&“) abschließt

```
bash > xclock -upd 1 -bg red &
[2] 7662
bash >
```

Am Anfang der Ausgabezeile steht eine eingeklammerte „2“. Das bedeutet, dass bereits eine Anwendung im Hintergrund läuft, die vom aktuellen Terminal aus gesteuert wird. Die Zahl dahinter ist die Prozessnummer, unter der das Betriebssystem die zweite xclock verwaltet.

Das eingebaute Shellkommando „fg“ holt ein im Hintergrund laufendes Kommando wieder in den Vordergrund

Wenn man nun der Shell wieder die Kontrolle über das Programm xclock geben möchte, kann man das mit dem eingebauten Shellkommando „fg“ („foreground“) tun

```
bash > fg
xclock -upd 1 -bg red
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Die Ausgabezeile nennt das Kommando, das nun wieder „im Vordergund“ läuft. Die Eingabeaufforderung der Shell ist nun wieder weg und Tastatureingaben werden direkt an die zweite xclock weitergeleitet. Wenn man beispielsweise die Tastenkombination Ctrl-c drückt, verschwindet die xclock und die Shell übernimmt wieder das Kommando über das Terminal (-fenster)[33]. Genauso könnte man nun auch die erste xclock beenden.

Die Tastenkombination Ctrl-c beendet ein laufendes Programm.

Dieses Verfahren ist etwas mühsam, wenn man eigentlich nur die erste im Hintergrund laufende Anwendung beenden, die anderen aber weiter laufen lassen möchte. Um das zu demonstrieren, wird zunächst wieder eine zweite xclock gestartet, diesmal mit goldgelbem Hintergrund

```
bash > xclock -upd 1 -bg gold &
[2] 7915
bash >
```

Anzumerken ist, dass die Prozessnummer (und die Hintergrundfarbe der Uhr) eine andere ist. Sonst ist alles wie vorher.

Will man wissen, welche im Hintergrund laufenden Anwendungen vom aktuellen Terminal kontrolliert werden, gibt man „jobs“ ein

```
bash > jobs
[1]-  Running                 xclock -upd 1 &
[2]+  Running                 xclock -upd 1 -bg gold &
bash >
```

In diesem Fall sind es natürlich die beiden xclocks. Beide laufen. Die Zahl in Klammern ist die so genannte „Job ID“, die man nicht mit der Prozessnummer („Process ID“) verwechseln darf. Die Process ID ist systemweit eindeutig. Die Job ID’s sind nur innerhalb der Hintergrundprozesse eindeutig, die von einem Terminal aus gesteuert werden.

---

[33] Das funktioniert fast immer – nur nicht, wenn man es dringend bräuchte, wenn das Programm nämlich nicht mehr reagiert. Aber dann gibt es andere Mittel und Wege (s. Abschnitt 6.3 auf Seite 44).

Um nun die erste xclock gezielt wieder in den Vordergrund zu holen, gibt man dem fg-Kommando einfach die Job ID als Argument mit. In diesem Fall also

```
bash > fg 1
xclock -upd 1
```

Gibt man fg keinen Parameter mit, wird derjenige Job in den Vordergrund geholt, der in der Ausgabe von jobs das „+“ hinter seiner Job ID hat.

Gibt man jetzt noch einmal „jobs“ ein, so kann man feststellen, dass die Job ID gleich geblieben ist, obwohl nur noch ein Hintergrundprozess übrig ist.

Um ein Hintergrundprogramm zu beenden, muss man es nicht erst in den Vordergrund holen. Das Kommando `kill %<Job ID>`[34] tut genau das, was der Name erwarten lässt.

9.4 Shell History

Hat man ein Kommando mit all seinen Optionen und Parametern einmal eingetippt und möchte es noch ein zweites Mal mit leicht veränderten Parametern starten, muss man nicht die ganze Arbeit wiederholen: Die bash merkt sich Kommandos. Durch mehrfaches Drücken der „Pfeil nach oben“-Taste erscheinen in umgekehrter Reihenfolge die Kommandos, die man bereits in der aktuellen Shell abgesetzt hat. Wenn man etwa zwei Terminalfenster („xterm“ oder „konsole“) in der oberen linken und der oberen rechten Ecke des Bildschirms starten möchte ...

[34] Die Job-ID ist die Zahl in eckigen Klammern am Anfang jeder Ausgabezeile von „jobs“

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

```
bash > xterm -geometry +0+0 &[35]
[1] 8017
bash > ... Pfeil-oben drücken ... xterm -geometry -0+0 &
[2] 8018
bash >
```

Durch einmaliges drücken der Pfeil-oben-Taste erscheint der erste Befehl wieder. Nun kann man mit der Pfeil-Links-Taste nach links wandern und das „+“ vor der ersten Null in ein „-” verwandeln. Nun noch die Enter-Taste drücken und das zweite Kommando wird ausgeführt.

Beendet man eine Shell und die Umgebungsvariable `$HISTFILE` ist gesetzt[36] (standardmäßig ist sie auf `$HOME/.bash_history` gesetzt), so werden alle Kommandos der abgeschlossenen Sitzung an diese Datei angehängt. Erst jetzt kann man die Kommandos auch von anderen Teminals aus mit der „Pfeil-oben“ Taste erreichen.

Auf Rechnern, die von vielen verschiedenen Menschen benutzt werden, stellt die Datei `.bash_history` eine gewisse Sicherheitslücke dar, weil in ihr alle Kommandos gespeichert sind, die man in letzter Zeit abgesetzt hat. Unter Umständen sind sogar Passwörter in dieser Datei nachzulesen. Im Zweifelsfall sollte man die Variable `$HISTFILE` auf einen leeren String setzen, zum Beispiel dadurch, dass man die Zeilen

```
HISTFILE=""
export HISTFILE
```

---

[35] Die `-geometry` Option kennen alle X-Windows-Anwendungen. eigentlich verlangt sie ein Argument der Form „<Breite>x<Höhe>[+|-]<xpos>[+|-]<ypos>“. <Breite> und <Höhe> geben die Größe des Fensters an. Meistens (wie zum Beispiel bei xclock) in Pixeln. Bei Terminalfenstern und ähnlichen Programmen aber in Zeichen. <xpos> und <ypos> geben die Position des Fensters (in Pixeln) an. wenn vor der Zahl ein „+“ steht, wird der Abstand des Fensters vom linken bzw. oberen Bildschirmrand definiert. Wenn vor der Zahl ein „-” steht, wird der Abstand des Fensters vom rechten bzw. unteren Rand des Bildschirmes definiert. Der Ausdruck „60x12+120-60“ bedeutet also ein Fenster der Größe 60 mal 12 Pixeln bzw. Zeichen, das 120 Pixel vom linken und 60 Pixel vom unteren Bildschirmrand entfernt positioniert wird. Lässt man die Größe oder die Position weg, werden Standardwerte verwendet.

[36] Was man leicht mit dem Kommando „`echo $HISTFILE`“ überprüfen kann

in die entsprechenden Startup-Dateien einbaut. Man verzichtet damit lediglich auf den Komfort, Kommandos aus früheren Sitzungen wieder hervorholen zu können.

Es gibt noch raffiniertere Methoden, auf den Inhalt der Shell-History zuzugreifen

Immer dran denken: Lesen nützt nix. Probieren Sie es aus!!!!

`history`	Dieses Kommando fürhrt dazu, dass alle Kommandos der Shell-History nummeriert ausgegeben werden
`!<n>`	Startet das Kommando mit der Nummer <n> in der History noch einmal
`!-<n>`	Startet das <n>.-letzte Kommando der History noch einmal
`!!`	Startet das letzte Kommando noch einmal
`!?<string>?`	Startet das letzte Kommando, in dem die Zeichenkette „<string>“ vorkam noch einmal

```
bash > !?xclock?
xclock -upd 1 -bg gold &
bash >
```

`!!:s/bla/blubb/`	Startet das letzte Kommando noch einmal, wobei die Zeichenkette „bla“ durch „blubb“ ersetzt

```
bash > !!:s/gold/blue/
xclock -upd 1 -bg blue &
bash >
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Fragen zur Sebstkontrolle

1. Was bedeutet es, wenn ein Kommando im Hintergrund ausgeführt wird?

2. Was bewirkt die Tastenkombination Ctrl-z?

3. Was bewirkt die Tastenkombination Ctrl-c?

4. Wie bekommt man eine Liste der aktuell im Hintergrund laufenden Prozesse, die vom aktuellen Terminal aus gesteuert werden?

5. Wie kann man das letzte Kommando am schnellsten noch einmal starten?

Die Antworten finden Sie auf Seite 98. Aber versuchen Sie in jedem Fall erst einmal, selbst auf die Lösung zu kommen und schauen Sie im Zweifelsfall noch einmal im Text nach. Durch „Schummeln“ schaden Sie niemanden – außer sich selbst.

10 Shellskripte

Die bash oder jede andere UNIX-Shell ist nicht nur ein Kommandozeileninterpreter, um einzelne Kommandos abzusetzten, sondern stellt auch eine Programmiersprache zur Verfügung, mit der man aus vorhandenen UNIX-Kommandos neue Funktionalitäten realisieren kann, so wie das in der Windows-Welt mit Batch-Dateien möglich ist. Diese Programme, die in der UNIX-Welt als Shellsripte bezeichnet werden, werden nicht mit einem Compiler in ausführbaren Maschinencode übersetzt, sondern zur Laufzeit interpretiert. Der Interpreter ist die gleiche Shell, die auch einzelne Kommandos des Benutzers oder der Benutzerin entgegen nimmt.

Wie jede ordentliche Programmiersprache bietet die bash

- Variable
- Ein- und Ausgabe
- Schleifen
- Bedingte Verzweigungen
- Funktionen

Ein Shellskript ist eine Textdatei, die eine Reihe von Anweisungen enthält, die die Shell, z.B. die bash, ausführen kann. Die einzelnen Anweisungen in einem Shellskript werden entweder durch einen Zeilenumbruch oder durch ein Semikolon (;) getrennt. Sie werden in der Reihenfolge abgearbeitet, in der sie in der Datei stehen.

10.1 Mein erstes Shellskript

```
1:    #!/bin/bash
2:    # --- Dies ist ein Kommentar
3:
4:    echo „Hallo Welt!“
5:    /bin/date
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Dieses einfache Shellskript beginnt mit zwei Kommentarzeilen. Alles was in einer Zeile hinter einem *Hash* (#) steht, wird von der Shell nicht interpretiert. Die Zeichenfolge #! Am Anfang der ersten Zeile heißt „*Shebang*“[37]. Diese Zeichenkombination führt dazu, dass die Shell den Rest der Zeile als Aufruf eines geeigneten Interpreters für den Rest der Datei interpretiert.

Es gibt unterschiedliche Wege, ein Shellskript zu starten. Normalerweise wird das Skript nicht von der Shell ausgeführt, in der es gestartet wurde, sondern in einer Subshell. Das lässt sich einfach mit folgendem Skript testen:

```
1:    #!/bin/bash
2:    #
3:    echo "Meine Prozess-ID ist $$"
```

Die Variable $$ steht für die Prozess-ID des aktuellen Prozessen, in diesem Fall der aktuellen Shell. Ein Weg das Skript zu starten ist

```
bash > echo $$
67802
bash > bash meinskript
Meine Prozess-ID ist 67843
bash >
```

Die Prozess ID der Shell, in der das Skript aufgerufen wird, ist in der Tat eine andere als die der Shell, die das Skript ausführt.

Eine andere Möglichkeit, ein Shellskript auszuführen, ist die Datei ausführbar zu machen und dann wie ein compiliertes Programm auszuführen

[37] Die Herkunft dieses Begriffs ist nicht ganz klar. Eine Theorie besagt, dass es sich um eine Abkürzung für *„Sharp (#) Bang (!)“* handelt.

```
bash> chmod u+x meinskript
bash> echo $$
67802
bash> ./meinskript
Meine Prozess-ID ist 67850
```

Auch hier wird das Skript in einer Subshell ausgeführt. Manchmal möchte man ein Shellskript aber auch in der aktuellen Shell ausführen. Dazu muss das Shellsript „gesourced“ werden

```
bash> echo $$
67802
bash> . meinskript
Meine Prozess-ID ist 67802
```

Das wird zum Beispiel in der Datei `.bashrc` im Heimatverzeichnis eines Benutzers gemacht, um die Anweisungen in der systemweiten Datei (`/etc/bashrc`) auszuführen.

Innerhalb eines Shellskriptes kann man eine Sequenz von Anweisungen in einer Subshell ausführen, indem man diese Anweisungen in runde Klammern „`(...)`“ einschließt.

10.2 Variable

Variable werden definiert durch

```
VARNAME=“Wert“
```

Beim Zugriff auf die Variable wird ein Dollarzeichen vorangestellt und der Name der Variablen in geschweifte Klammern eingeschlossen

```
echo ${VARNAME}
Wert
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Die geschweiften Klammern kann man in vielen Fällen weglassen[38]. Es gibt noch ein paar Varianten beim Zugriff auf Variable

`${VARNAME:-wert}`	$VARNAME, wenn definiert, sonst „wer“
`${VARNAME:=wert}`	Falls $VARNAME nicht definiert ist, wird zunächst „wert“ zugewiesen
`${VARNAME:?Fehler}`	Falls $VARNAME nicht definiert ist, wird „Fehler“ ausgegeben und die Prozedur abgebrochen
`${#VARNAME}`	Gibt die Länge der Variablen $VARNAME aus
`$(Kommando)`	Liefert die Ausgabe (stdout) von Kommando

Eine weitere Möglichkeit, die Ausgabe eines Kommandos z.B. in eine Variable umzulenken sind „Backquotes“ (`) (auf der Deutschen Tastatur links neben der Backspace-Taste in Kombination mit der Umschalttaste

```
MYUID=`grep ${UID} /etc/passwd | cut -d ,:' -f 1
```

Die Man-Page der bash beschreibt unter der Überschrift *„Parameter Expansion“* noch eine Vielzahl von Möglichkeiten, Shellvariable zu formatieren oder zu bearbeiten.

Eine Reihe wichtiger Variablen ist bereits vordefiniert (nur ein Auszug):

`$PWD`	Pfad des aktuellen Verzeichnisses
`$UID`	(numerische) User-ID des aktuellen Benutzers
`$USER`	Name des aktuellen Benutzers

[38] Wenn bei einem Shellskript unerklärliche Fehler auftreten, könnte das u.U. daran liegen, dass Variablennamen nicht in geschweifte Klammern eingefügt sind. Das kann vor allem dann vorkommen, wenn direkt hinter dem Variablennamen kein Leerzeichen, sondern ein anderes Zeichen folgt.

$HOME	Pfad des Heimatverzeichnisses des aktuellen Benutzers
$PATH	Pfade, unter denen nach ausführbaren Dateien gesucht wird
$0	Name, unter dem das Skript aufgerufen wurde
$1 - $9	Kommandozeilenparameter 1 - 9
$*	Alle Kommandozeilenparameter in einem String
$@	Alle Kommandozeilenparameter in separaten Strings
$#	Zahl der Kommandozeilenparameter
$$	Prozess-ID des aktuellen Prozesses
$?	Rückgabewert des letzten Kommandos

10.3 Arithmetische Ausdrücke

In Shellskripten kann man auch rechnen. Dazu gibt es zwei Kommandos: `let` und `expr`.

Das eingebaute Shellkommando `let` erlaubt alle üblichen Rechenoperationen (s. `man bash`), zum Beispiel

```
bash> a=3
bash> let y=a+2  # Merke: keine Leerzeichen!
bash> echo $y
5
```

Das externe Kommando `/bin/expr` liefert das Ergebnis einer arithmetischen Operation auf den stdout-Kanal:

```
bash> a=3
bash> expr $a + 2
5
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

10.4 Verkettung von Anweisungen

Die bash bietet verschiedene Möglichkeiten, mehrere Kommandos miteinander zu verketten

`cmd1 ; cmd2`	cmd2 wird ausgeführt, wenn cmd1 abgeschlossen ist
	`mv datei1 tmpdat; mv datei2 datei1; \`
	`mv tempdat datei1`

`cmd1 && cmd2`	cmd2 wird ausgeführt, wenn cmd1 erfolgreich abgeschlossen ist `cp datei1 /tmp && rm datei1`
`cmd1 || cmd2`	cmd2 wird ausgeführt, wenn cmd1 mit einem Fehler abgeschlossen ist `cp datei1 /tmp || echo „Fehler!“`
`cmd1 | cmd2`	Die Standardausgabe von cmd1 wird zur Standardeingabe von cmd2 `ls /usr/lib | more`

10.5 Logische Ausdrücke

Jede Programmiersprache, die etwas auf sich hält, erlaubt die Programmierung von bedingten Verzweigungen. Dazu muss zunächst entschieden werden, welchen Weg das weitere Skript nehmen soll. Dazu braucht man logische Ausdrücke. In der bash steht die zahl 0 für *wahr* und die Zahl 1 für *falsch*[39].

Das Kommando `/usr/bin/true` tut nichts anderes, als einen Rückgabewert von 0 („*wahr*“) zu liefern. Umgekehrt liefert das Kommando `/usr/bin/false` den Rückgabewert 1 („*falsch*“).

Das Kommando `/bin/test` wertet den Ausdruck, der als Kommandozeilenparameter übergeben wird. Ist der Ausdruck wahr, liefert

[39] Also gerade anders herum als in der Programmiersprache C!

test einen Rückgabewert von 0, andernfalls einen Rückgabewert von 1.

Zum Vergleich numerischer Werte werden folgende Ausdrücke verwendet:

`n1 -eq n2`	n1 = n2 (equal)
`n1 -ne n2`	n1 != n2 (not equal)
`n1 -gt n2`	n1 > n2 (greater than)
`n1 -ge n2`	n1 >= n2 (greater or equal)
`n1 -le n2`	n1 <= n2 (less or equal)
`n1 -lt n2`	n1 < n2 (less than)

Diese wie auch alle folgenden logischen Ausdrücke können auf zwei verschiedene Weisen verwendet werden. Entweder, indem man das Kommando test verwendet

```
bash> test 4 -gt 2
bash> echo $? # Rueckgabewert des test-Kommandos
0
```

Alternativ kann man den logischen Ausdruck in mit eckigen Klammern einrahmen. Die Shell interpretiert das genauso wie das test-Kommando

```
bash> [ 4 -gt 5 ]
bash> echo $? # Rueckgabewert des test-Kommandos
1
```

Zum Vergleich von Strings werden dagegen die mathematischen Zeichen verwendet:

`[ -z $a ]`	Wahr, wenn Zeichenkette $a leer
`[ -n $a ]`	Wahr, wenn Zeichenkette $a nicht leer ist

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

`[ $a ]`	Wahr, wenn Zeichenkette $a nicht leer
`[ $a = $b ]`	Wahr, wenn beide Zeichenketten gleich sind
`[ $a != $b ]`	Wahr, wenn beide Zeichenketten ungleich sind
`[ $a < $b ]`	Wahr, wenn $a lexikalisch vor $b kommt
`[ $a > $b ]`	Wahr, wenn $a lexikalisch nach $b kommt

Schließlich können noch logische Operationen auf Dateien angewandt werden:

`[ -e dateiname ]`	Wahr, wenn die Datei existiert
`[ -s dateiname ]`	Wahr, wenn die Datei existiert
`[ -x dateiname ]`	Wahr, wenn die Datei existiert und ausführbar ist
`[ -f dateiname ]`	Wahr, wenn die Datei existiert und eine reguläre Datei ist
`[ -d dateiname ]`	Wahr, wenn die Datei existiert und ein Verzeichnis ist
`[ ! -e dateiname ]`	Wahr, wenn die Datei nicht existiert

10.6 Bedingte Verzweigungen

In einem Shellskript können abhängig vom Wert eines logischen Ausdrucks verschiedene Kommandos ausgeführt werden.

Beispiel: zu Beginn eines Shellskripts soll geprüft werden, ob die Zahl der übergebenen Kommandozeilenparameter korrekt ist.

```
#! /bin/bash
#
if [ $# -ne 3 ]; then
  echo „Das Skript erfordert drei Parameter!“
  exit 1
fi
...
```

Wichtig: das Schlüsselwort `then` muss entweder auf einer neuen Zeile oder hinter einem Semikolon stehen. Die Kommandosequenz, die nur bedingt ausgeführt werden soll, endet mit dem Schlüsselwort `fi`, das auch wieder auf einer neuen Zeile stehen muss.

Man kann auch mehrere mögliche Pfade unterscheiden

```
if [ $# -eq 0 ]; then
  echo „Kein Argument uebergeben“
elif [ $# -eq 1 ]; then
  echo „Genau ein Argument uebergeben“
else      # wird in allen anderen Faellen ausgeführt
  echo „Mehr als ein Argument uebergeben“
fi
```

Im Prinzip kann man als logischen Ausdruck jedes beliebige Kommando verwenden. Ist der Rückgabewert 0 (was normalerweise einem fehlerfreien Programmablauf entspricht), wird der „if“-Zweig ausgeführt, sonst der „else“-Zweig (wenn vorhanden).

Wenn man deutlich mehr als zwei Fälle unterscheiden will, ist die case-Anweisug eine interessante Alternative zum `if ... then ... else` Konstrukt

```
case $TERM in
xterm)
    # Konfiguration für X11-Terminalemulation
    ...
    ;;
vt100)
    # Konfriguration für DEC VT100 Terminals
    ...
    ;;
*)
    # Konfiguration für alle anderen Terminaltypen
    ...
    ;;
esac
```


© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Jeder Zweig wird mit einem doppelten Semikolon, das gesamte Konstrukt mit dem Schlüsselwort esac abgeschlossen

10.7 Schleifen

Die häufigste Form der Schleife ist das `for ... in` Konstrukt, das zum Beispiel häufig dazu verwendet wird, die Liste der Kommandozeilenparameter abzuarbeiten

```
ARGC=1
for ARG in $@; do
  echo „Argument ${ARGC} : ${ARG}“
  let ARGC=ARGC+1
done
```

Die Kommandos, die mehrfach ausgeführt werden sollen, werden von den Schlüsselwörtern do und done eingerahmt, die wieder auf einer eigenen Zeile oder hinter einem Semikolon stehen müssen.

Die bash bietet auch eine `for`-Schleife, die so ähnlich funktioniert, wie in der Programmiersprache C, die aber kein Shell-Standard ist

```
for (( i = 0; $i < 10; i = $i + 1 )); do
  echo „Das ist der ${i}-te Durchgang“
done
```

Neben den for-Schleifen, deren Ausführung nach einer bestimmten Zahl von Durchläufen abbricht, gibt es noch zwei Typen von Schleifen, deren Ausführung abgebrochen, wenn ein logischer Ausdruck ein bestimmtes Ergebnis hat.

Die erste Version führt den Inhalt der Schleife aus, so lange der logische Ausdruck wahr ist

```
i=0
while [ $i -lt 10 ];do
  echo „while: Durchgang ${i}“
```

```
  i=`expr $i + 1`
done
```

Die zweite Version führt den Inhalt der Schleife aus, so lange der logische Ausdruck falsch ist

```
i=10
until [ $i -lt 10 ];do
  echo „until: Durchgang ${i}“
  i=`expr $i - 1`
done
```

Eine Schleife muss nicht unbedingt bis zu ihrem „natürlichen Ende“ ausgeführt werden.

```
for i in $liste: do
  if [ ... ]; then
    break
  fi
  while [ ... ]; do
    if [ ... ]; then
      break
    elif [ ... ]; then
      break 2
    fi
  done
done
```

Die break-Anweisung dient dazu, Schleifen abhängig von einem logischen Ausdruck abzubrechen. Mit einem zusätzlichen Argument kann man auch verschachtelte Schleifen verlassen

Will man die Schleife nicht abbrechen, sondern nur in die nächste Iteration springen, ohne den Rest des Schleifeninhalts abzuarbeiten, bietet sich die continue-Anweisung an, die man analog zur break-Anweisung verwendet.

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

Nicht nur die Schleife, sondern die ganze Shell wird durch die `exit`-Anweisung beendet. Gibt man der Anweisung eine Zahl als Argument mit, so wird diese als Rückgabewert der beendeten Shell verwendet

```
bash > bash
bash2 > exit 4
bash > echo $?
4
```

10.8 Shell-Funktionen

Shell-Funktionen können überall dort stehen, wo auch andere Shellkommandos stehen dürfen. Die allgemeine Syntax zur Definition einer Shellfunktion ist

```
[ function ] name() { liste; }
```

Das Schlüsselwort `function` ist optional. Auf Argumente, die beim Funktionsaufruf übergeben werden sollen, kann über die bekannten Variablen $1, ... $9 zugreifen.

Eine Funktion, um alle Buchstaben einer Zeichenkette durch Großbuchstaben zu ersetzten, könnte man so definieren

```
function upper() {
  echo $1 | tr [a-z][ A-Z]
}
```

Der Aufruf der Funktion sieht dann so aus

```
upper beispiel
```

11 Antworten auf die Testfragen

Seite 28

1. Woran erkennt man eine absolute Pfadangabe in UNIX?
   An dem führenden Slash

2. Was bewirkt das Kommando `cd -` ?
   Einen Wechsel in das Verzeichnis, in dem man vor dem letzten cd-Kommando war

3. Mit welchem UNIX-Kommando können sie in Ihrem aktuellen, leeren Verzeichnis den Verzeichnisbaum `develop/java/src` erzeugen?
   mkdir -p develop/java/src

4. Wann funktioniert das Kommando `rmdir` nicht?
   Wenn das Verzeichnis, das gelöscht werden soll nicht leer ist

Seite 45

1. Erklären Sie in Stichworten den Unterschied zwischen einer Kopie, einem Hardlink und einem Symlink
   Kopie: Aus einer Datei werden zwei
   Hardlink: Ein zusätzlicher Verzeichniseintrag für einen existierenden i-node
   Symlink: Ein zusätzlicher Verzeichniseintrag, der auf einen anderen Verzeichniseintrag verweist

2. Wenn man mit dem Kommando mv ein Verzeichnis mit tausenden von Dateien verschiebt, geht das ganz schnell, wenn man es innerhalb einer Partition verschiebt, aber viel langsamer, wenn man es von einer Partition auf die andere verschiebt. Warum?
   Im ersten Fall wird nur ein neuer Verzeichniseintrag angelegt und der alte gelöscht. I-nodes und Daten werden nicht angerührt. Im zweiten Fall müssen die Daten tatsächlich kopiert werden

3. Wie kann man sich die aktuellste Datei in einem Verzeichnis ausgeben lassen?
   `ls -ltr <Pfad> | tail -1` oder
   `ls -lt | head -2 | tail -1`

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

4. Wie kann man die vorletzte Zeile einer Textdatei anzeigen lassen?
   tail -2 <Datei> | head -1

5. Was bewirkt die UNIX Kommandozeile
   „cmd 3>&2 2>&1 1>&3“?
   Der stdout- und der stderr-Kanal werden vertauscht

Seite 70

1. Mit welcher UNIX Kommandozeile können Sie feststellen, auf welcher Festplatte das Verzeichnis /home/musterma liegt?
   df /home/musterma

2. Mit welcher UNIX Kommandozeile könne sie feststellen, wie viel Platz das Verzeichnis /data/mp3 auf der Festplatte in Anspruch nimmt?
   du -sk /data/mp3 oder du -sh /data/mp3

3. Mit welchem erweiterten regulären Ausdruck können Sie in einer Datei Daten im ISO 8601-Format (Vier Ziffern, Bindestrich, zwei Ziffern, Bindestrich, zwei Ziffern, Beispiel: 2006-12-31) finden?
   [0-9]{4}-[0-9]{2}-[0-9]{2}

4. Warum funktioniert dieser reguläre Ausdruck nicht in der Kommandozeile „grep '<regexp>' datei“, wobei <regexp> für die Antwort auf Frage 3 steht?
   Weil grep in dieser Form normale reguläre Ausdrücke erwartet

5. Was müssen Sie tun, damit die Kommandozeile funktioniert?
   Entweder die geschweiften Klammern maskieren oder „egrep“ bzw. „grep -E“ als Kommando verwenden

6. Geben Sie die UNIX Kommandozeile an, mit der Sie in einer Textdatei mehrere aufeinander folgende Leerzeilen durch eine einzige und mehrere aufeinander folgende Leerzeichen durch ein einziges ersetzen können. Um diese Frage zu beantworten, müssen Sie eventuell die man-Pages zu den Befehlen cat und tr studieren!
   cat -s <Datei> | tr -s ' '

7. Mit welchem sed-Kommando können Sie aus einem Text alle Zeilen löschen, die mit einem „#“ beginnen?
   sed –e ’/^#/d’ <Datei>

8. Mit welchem sed-Kommando können Sie in einem Text alle Daten in dem in Deutschland gebräuchlichen Format („24.12.2006“) in das ISO 8601-Format („2006-12-24“) verwandeln?
   sed –e ’s/\([0-9]\{2\}\)\.\([0-9]\{2\}\)\.\([0-9]\{4\}\)/\3-\2-\1/g’ <Datei>

Seite 76

1. Welche drei Varianten der bash Shell gibt es?
   nicht-interaktive, interaktive und Login-Shells

2. Woran erkennt man eine interaktive Shell?
   Das Kommando tty liefert den Namen des Terminals, mit dem die Shell verknüpft ist. Die Optionen der Shell enthalten die Option –i (Ausgabe von echo $- enthält ein „i“).

3. Wie kann man gezielt eine Login-Shell starten?
   Mit bash --login

4. Welche Startup-Datei(en) wird/werden ausgeführt, wenn man eine interaktive bash startet?
   Standardmäßig nur $HOME/.bashrc

Seite 83

1. Was bedeutet es, wenn ein Kommando im Hintergrund ausgeführt wird?
   Das Kommando ist nicht mit einem Terminal verbunden, d.h. es können keine Ein- oder Ausgaben gemacht werden

2. Was bewirkt die Tastenkombination Ctrl-z?
   Der aktuelle Prozess wird angehalten

3. Was bewirkt die Tastenkombination Ctrl-c?
   Der aktuelle Prozess wird beendet

4. Wie bekommt man eine Liste der aktuell im Hintergrund laufenden Prozesse, die vom aktuellen Terminal aus gesteuert werden?
   Mit dem Kommando „jobs“

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

5. Wie kann man das letzte Kommando am schnellsten noch einmal starten?
   Mit dem Kommando „!!“ oder durch drücken der Tasten „Pfeil oben“ und „Enter“

12 Index

;

; · 89

/

/dev/null · 44

&

& (Ampersand) · 78
&& · 89

>

> · 41
>> · 41

|

| · 89
| (Pipe) · 43
|| · 89

A

Arithmetische Ausdrücke · 88
awk · 65

B

Backquotes · 87
Bedingte Verzweigungen · 91
  case · 92
  if ... then · 91
bg · 78

C

cat · 35
cd · 24
chsh · 50
continue · 94
cp · 29
cut · 58

D

Dateisystem
  hierarchisches · 21
  Index-node · 29
df · 47
Dienstprogramme · 17
du · 48

E

egrep · 57
exit · 95

F

false (Anweisung) · 89
fg · 78
fgrep · 57
file · 39
function · 95

G

grep · 57

H

Hardlink · 30

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen

head · 37
Hintergrundprozesse · 77
history · 82

I

I/O redirecton · 40
id · 50
info · 20

J

Jobkontrolle · 77
jobs · 79

L

less · 36
Link
  Hard- · 30
  symbolischer · 32
ln · 30
Logische Ausdrücke · 89
ls · 23

M

man · 17
mkdir · 26
more · 36
mount · 46
mv · 35

O

od · 38

P

passwd · 49
Passwort ändern · 49
pwd · 22

R

Reguläre Ausdrücke
  erweiterte · 53
Reguläre Ausdücke · 52
rm · 34
rmdir · 27
ROT13 Verschlüsselung · 59

S

Schleife
  break · 94
  for ... in · 93
  for (C-artig) · 93
  until · 94
  while · 93
sed · 60
  austauschen · 62
  substitute · 61
Shell
  .bash_history · 81
  History · 80
  interaktive · 72
  Login- · 73
  nicht-interaktive · 72
Shell-Funktionen · 95
sort · 55
Startup-Dateien · 74
  .bash_login · 75
  .bash_profile · 75
  .bashrc · 74
  .profile · 75
  /etc/bashrc · 76
  /etc/profile · 75
stderr · 41
stdin · 41
stdout · 41
Suchpfad · 16
Symlink · 32

T

tail · 37
Tastenkombination
  Ctrl-c · 79
  Ctrl-z · 77
`test` (Anweisung) · 89
Textkonsole · 12
Torvalds · 7
touch · 29
tr · 59
`true` (Anweisung) · 89

U

Umlenkung von Ein- und Ausgabe · 40
unmount · 47

V

Virtualisierung · 10

W

which · 39

© 2016 Prof. Dr. Peter Väterlein, Hochschule Esslingen